

Image Segmentation



- Thersholding
- Image segmentation and its categories
- Adaptive thresholding algorithm
- Optimal Otsu thresholding algorithm
- Mean-shift algorithm
- K-Mean algorithm | KNN classification
- Region-growing algorithm
- Split and merge algorithm

Thresholding

➤ Thresholding

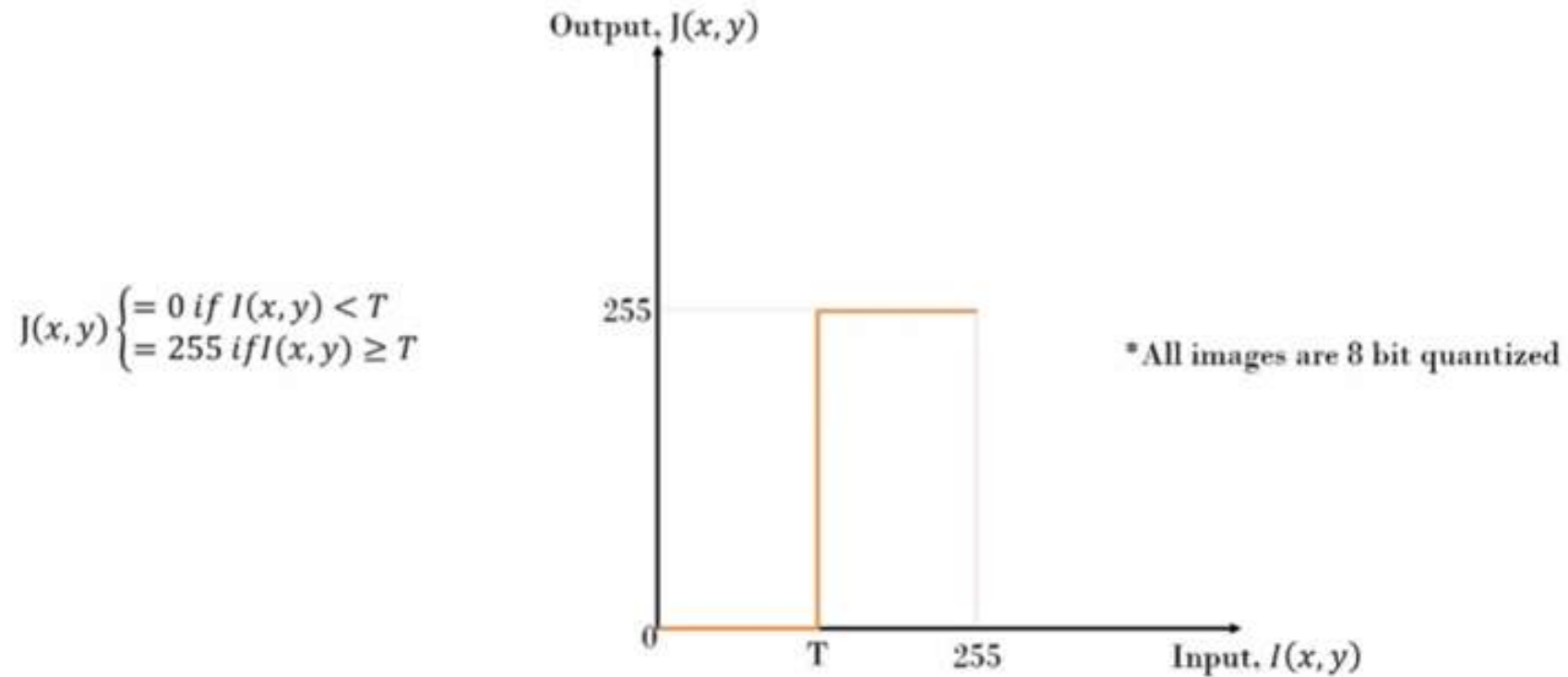


Image Segmentation: Thresholding

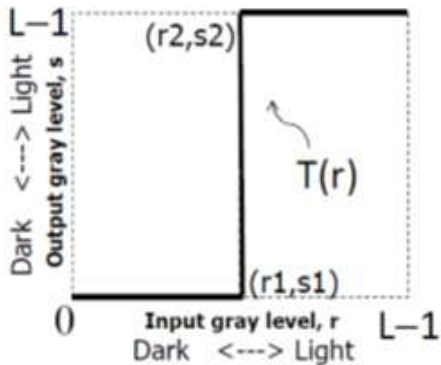
Thresholding is usually the first step in any segmentation approach



What is thresholding?

a digital image dividing it into subgroups is called segments

It reduces the overall complexity of the image and enabling the analysis and processing of each segment



Single value thresholding can be given mathematically as follows:

$$g(x, y) = \begin{cases} 1 & \text{if } f(x, y) > T \\ 0 & \text{if } f(x, y) \leq T \end{cases}$$

A threshold value (T) is defined.

Each pixel's intensity is compared to the threshold.

IF pixels with intensity greater than T are set to white ELSE set to black

Applications



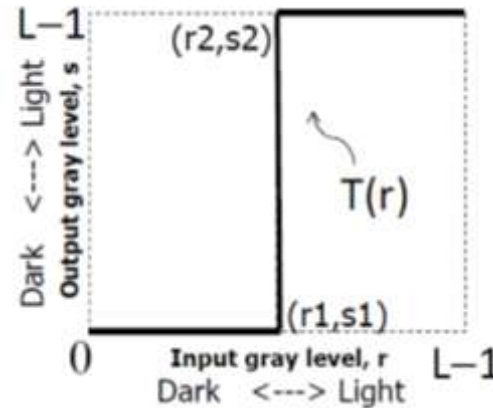
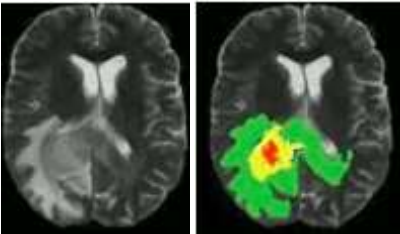
Object detection

Object identification in an image



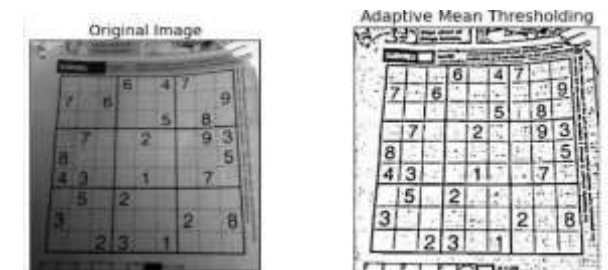
Image Segmentation

Separating object in an image



Optical character recognition (OCR)

Distinguish between the text and the background pixel in scanned document



Feature extraction

Isolating relevant feature in an image

Single Value Thresholding (SVT):

original grey image (0-255)

100	100	120	140
110	110	130	150
120	150	170	200
140	170	200	250

Total intensity=
 $100+100+120+\dots+250=2360$
 Average=
 $2360/16=147.5$

threshold image

0	0	0	255
0	0	255	255
0	255	255	255
255	255	255	255

Total intensity=
 $0+0+0+255+\dots+255=2550$
 Average=
 $2550/16=159.375$

Code:

```
threshold=128;
for j=1 to height
  for i=1 to width
    if (grey[j][i]<threshold) bw[j][i]=0
    else bw[j][i]=255;
```

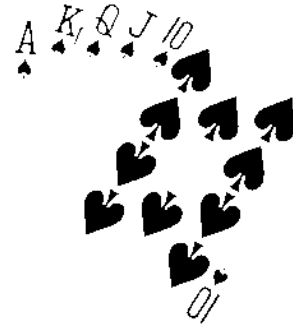


Example: SVT

Imagine a poker playing robot that needs to visually interpret the cards in its hand



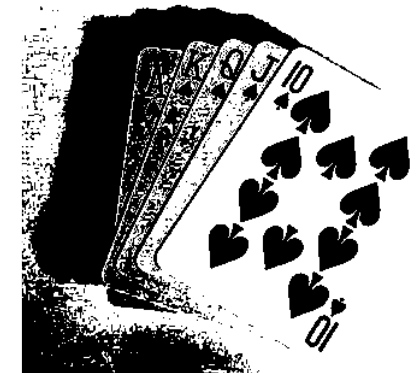
Original Image



Thresholded Image



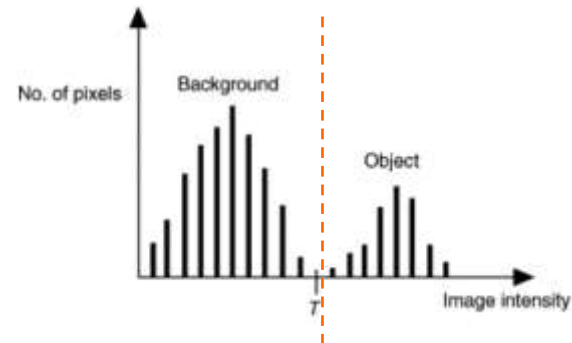
Threshold too low



Threshold too high

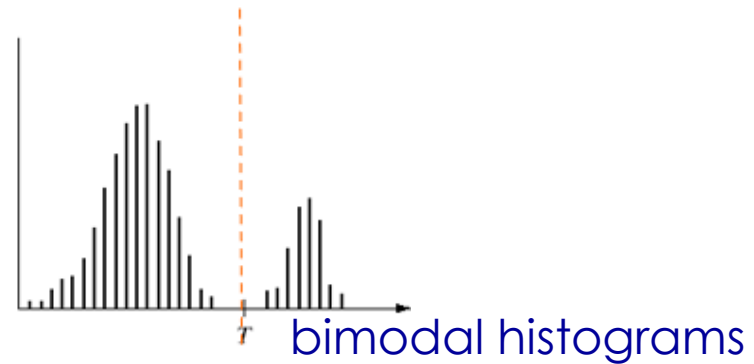
wrong threshold value, the results can be disastrous

Example

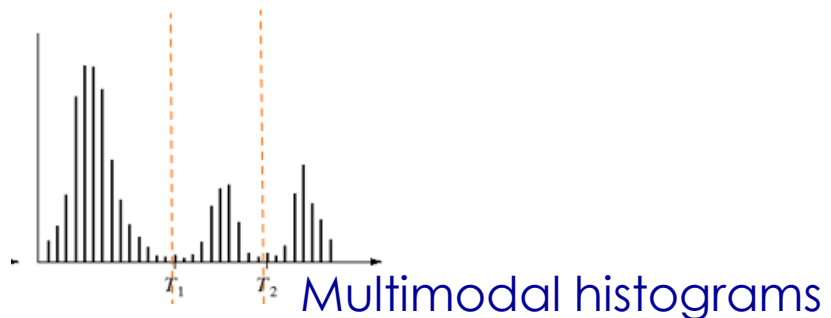


Problem with SVT

SVT only works for bimodal histograms.

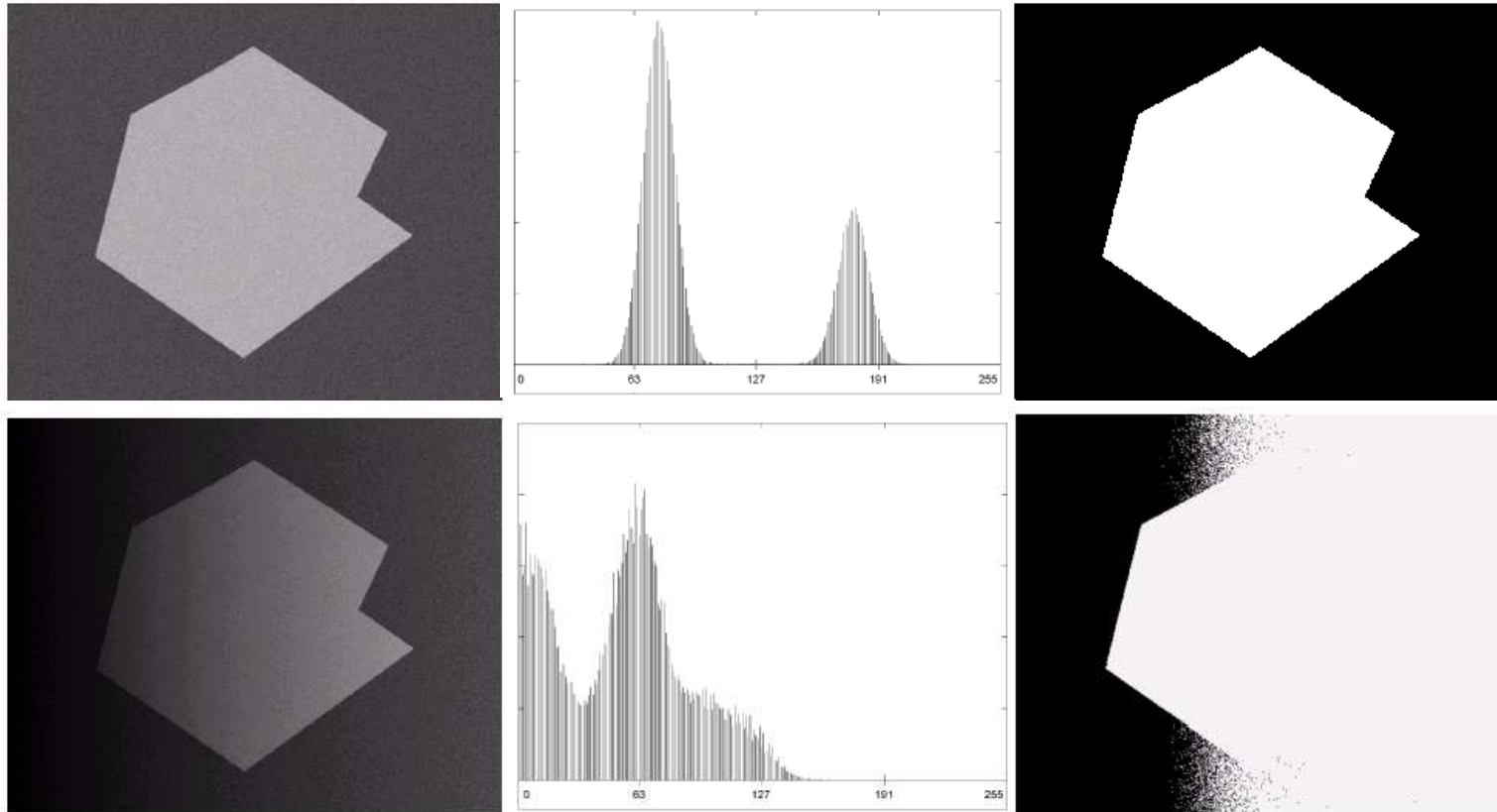


Images with multimodal histograms cannot use global thresholding.



Example:
we want to
isolate the contents
of the bottles,

SVT with Illumination and Solution



Uneven illumination can really upset a single valued thresholding scheme

Solution

An approach to handle the situation is to divide an image into sub images and threshold these individually

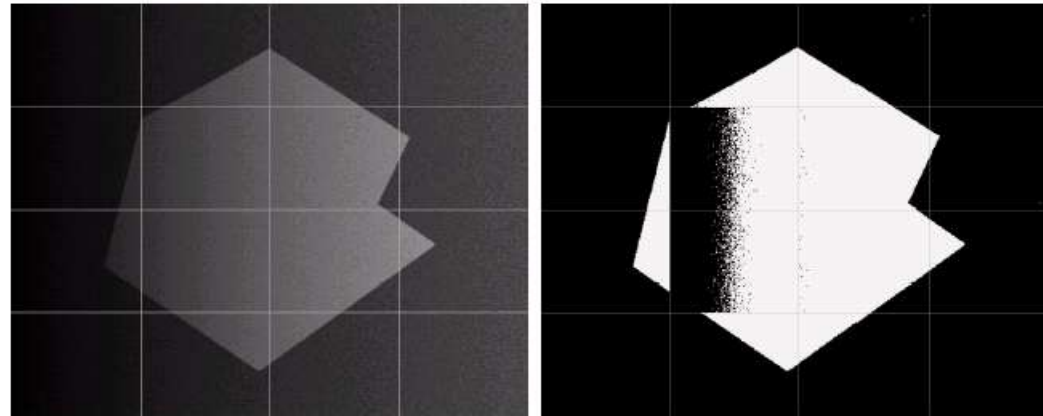
Mean Thresholding (T)

1. Select an initial estimate for T (typically the average grey level in the image)
2. Segment the image using T to produce two groups of pixels: G_1 consisting of pixels with grey levels $>T$ and G_2 consisting pixels with grey levels $\leq T$
3. Compute the average grey levels of pixels in G_1 to give μ_1 and G_2 to give μ_2
4. Compute a new threshold value

$$T = \frac{\mu_1 + \mu_2}{2}$$

5. Repeat steps 2 – 4 until the difference in T in successive iterations is less than a predefined limit T_∞

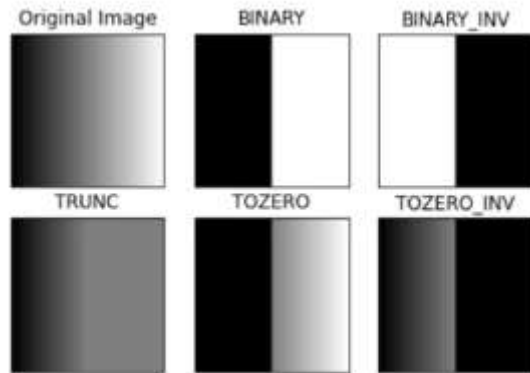
Mean Thresholding



As can be seen success is mixed

But, we can further subdivide the troublesome sub images for more success

Different Types of Thresholding



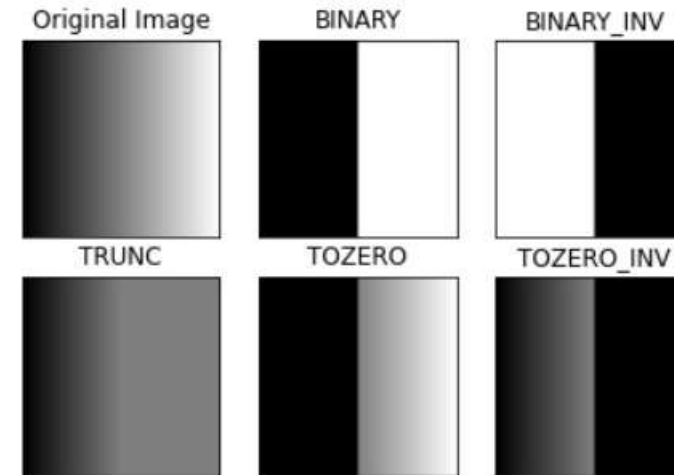
Type	Description
THRESH_BINARY	Pixel values above the threshold are set to 255, otherwise 0
THRESH_BINARY_INV	Pixel values above the threshold are set to 0, otherwise 255
THRESH_TRUNC	Pixel values above the threshold are set to the threshold value, otherwise unchanged
THRESH_TOZERO	Pixel values above the threshold are unchanged, otherwise set to 0
THRESH_TOZERO_INV	Pixel values above the threshold are set to 0, otherwise unchanged

cv2.threshold()

Parameters of OpenCV cv2.threshold() Method

The cv2.threshold() method in OpenCV requires several parameters to work properly. Here are the important points about the parameters needed:

Parameter	Description
src	The source image, which should be grayscale, will be used. Grayscale images are images that are black and white.
thresholdValue	This is the threshold value, which will be either above or below the pixel value. The threshold value will change based on the pixels.
maxValue	This will be the maximum value that can be assigned to a pixel.
threshold type	The threshold type is the method or technique that will be used on the image.

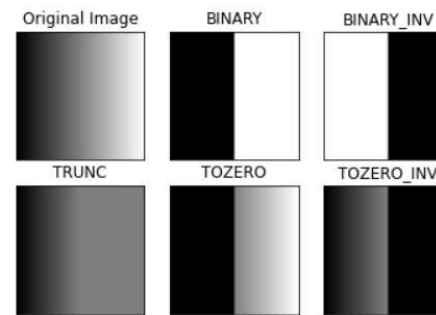


Return Value of OpenCV cv2.threshold() Method

The cv2.threshold() method in OpenCV returns the following:

- **retval:** The threshold value used for the operation.
- **dst:** The output image after the thresholding operation has been applied.
- **thresh** The threshold value that was passed as an argument to the method.
- **type:** The thresholding type that was used for the operation, which is specified as an argument to the method.

Different Types of Thresholding



```
import cv2 as cv
import numpy as np
from matplotlib import pyplot as plt

input_image = cv.imread('/content/Image(threshold).jpeg', cv.IMREAD_GRAYSCALE)

ret,thresh_image1 = cv.threshold(input_image,132,255,cv.THRESH_BINARY)
ret,thresh_image2 = cv.threshold(input_image,132,255,cv.THRESH_BINARY_INV)
ret,thresh_image3 = cv.threshold(input_image,132,255,cv.THRESH_TRUNC)
ret,thresh_image4 = cv.threshold(input_image,132,255,cv.THRESH_TOZERO)
ret,thresh_image5 = cv.threshold(input_image,132,255,cv.THRESH_TOZERO_INV)
titles = ['Input Image', 'BINARY', 'BINARY_INV', 'TRUNC', 'TOZERO', 'TOZERO_INV']
threshold_images = [input_image, thresh_image1, thresh_image2, thresh_image3, thresh_image4, thresh_image5]

for i in range(6):
    plt.subplot(1,6,i+1),plt.imshow(threshold_images[i], 'gray',vmin=0,vmax=255)
    plt.title(titles[i])
    plt.xticks([],plt.yticks([]))

plt.show()
```

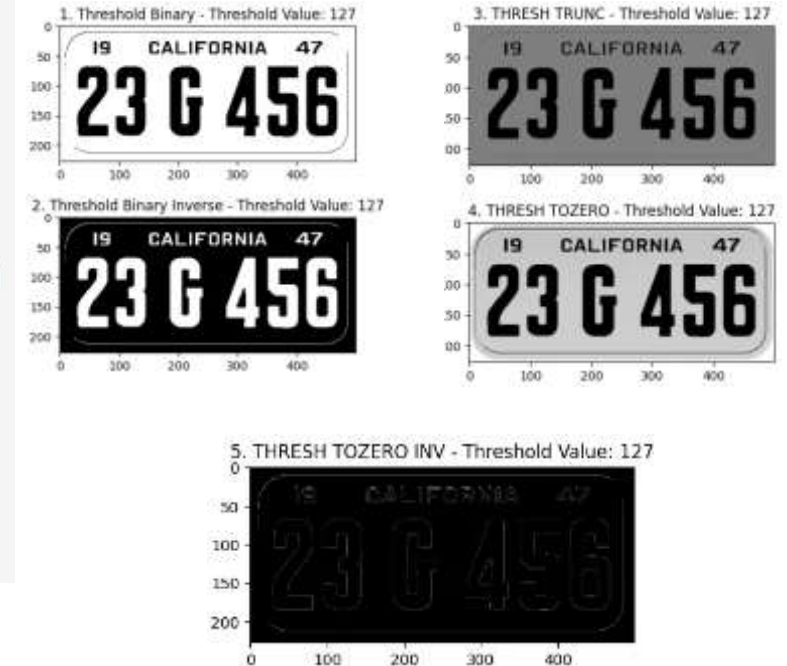


Image Segmentation

“Image segmentation” refers to dividing the image into groups of pixels based on some criteria.

A segmentation algorithm takes an image as input and outputs a collection of regions (or segments)

What are Superpixels?

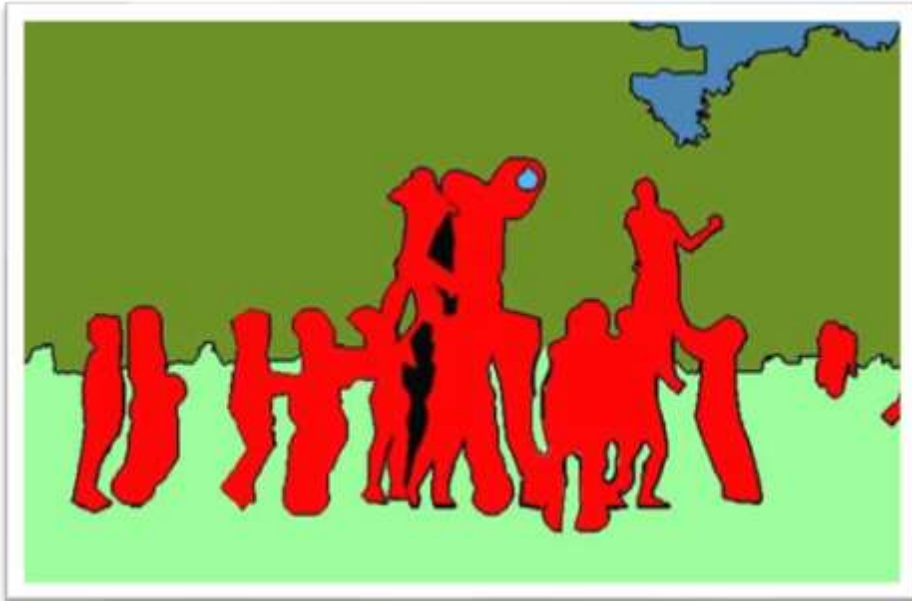
When we group pixels based on color, texture, or other low level primitives, we call these perceptual groups super pixels



the segmentation algorithm is simply grouping pixels of similar color and texture

What is Semantic Segmentation?

The goal is to assign a label to every pixel in the image.



The mask are represented as follows:

The people with red pixels

The grass is colored light green,

The trees are coded dark green

The sky is coded blue.

The pixels belong to the class “person” by simply checking if the mask color is red at that pixel

What is instance Segmentation?

Instance Segmentation is closely related to Object Detection.

the output is a mask (or contour) containing the object instead of a bounding box.

we are interested only in finding the boundaries of specific objects.



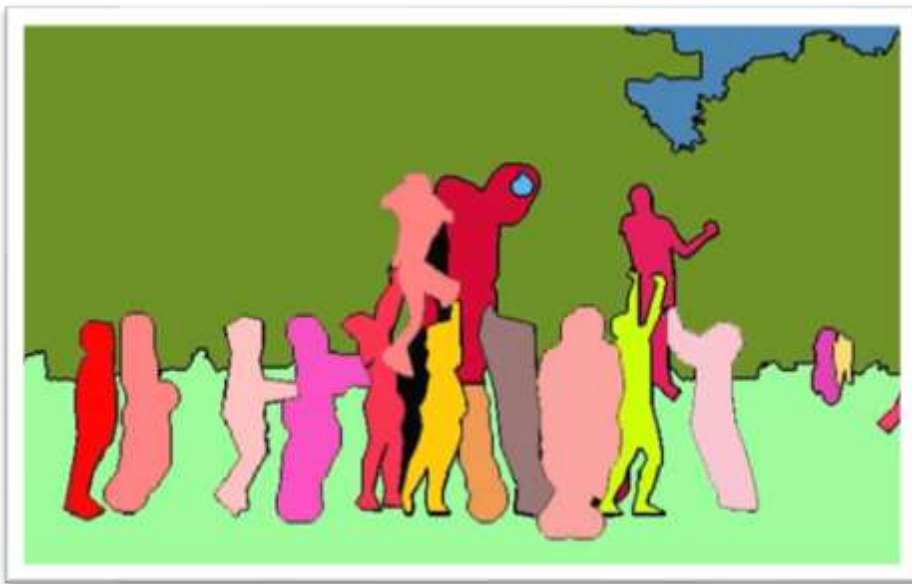
The output of an Instance Segmentation algorithm called Mask R-CNN

We see the mask for every person is colored differently so we can tell them apart.

What is Panoptic Segmentation?

Panoptic segmentation is the combination of Semantic segmentation and Instance Segmentation.

Every pixel is assigned a class (e.g. person), but if there are multiple instances of a class, we know which pixel belongs to which instance of the class..



Every pixel has a distinct color-coded label. the sky is coded blue, the trees are coded dark green, the grass is coded light green, and people are colored different shades of yellow, red and purple

The colors yellow and red both point to the same class — person — but two different instances of the same class.

We can tell different people apart by looking at the mask color.

Why Segmentation Is Essential in Robotics?

Enables robots to interpret complex environments

Supports safe navigation and collision avoidance

Allows precise manipulation of objects

Provides rich scene understanding for decision-making

Forms the basis for many higher-level AI tasks in robotics



Ameca Robot is real

Ameca is the world's most advanced human-shaped robot developed by a UK based company called Engineered Arts.

Ameca is designed to support the testing and development of artificial intelligence and machine learning systems

Application of Segmentation in Robotics

Object Recognition and Tracking

Robots use segmentation to identify and track objects in real time, enabling tasks like picking, sorting, and monitoring.

Scene Understanding (Semantic Segmentation)

Robots need to understand a scene environment—walls, floors, obstacles, humans—to operate safely.

Autonomous Navigation & Obstacle Avoidance

Segmentation helps robots detect obstacles, free space, and drivable areas, which is essential for mobile robots and autonomous vehicles.

Robot Grasping and Manipulation

To pick up or manipulate objects, a robot must know object boundaries and shapes. Segmentation isolates objects from the background for precise grasping..

Why Segmentation Is Essential in Robotics?

Advanced Perception & Visual Servoing

Segmentation supports perception-driven control, allowing robots to adjust their movements based on visual feedback.

Real-Time Scene Segmentation on Embedded Hardware

Robotics often requires segmentation models that run efficiently on low-power GPUs

Servoing refers to the act of using a feedback control system—a servomechanism—to automatically adjust, position, or control a mechanical device

Clustering

Clustering is the problem of grouping similar data points and represent them with a single token

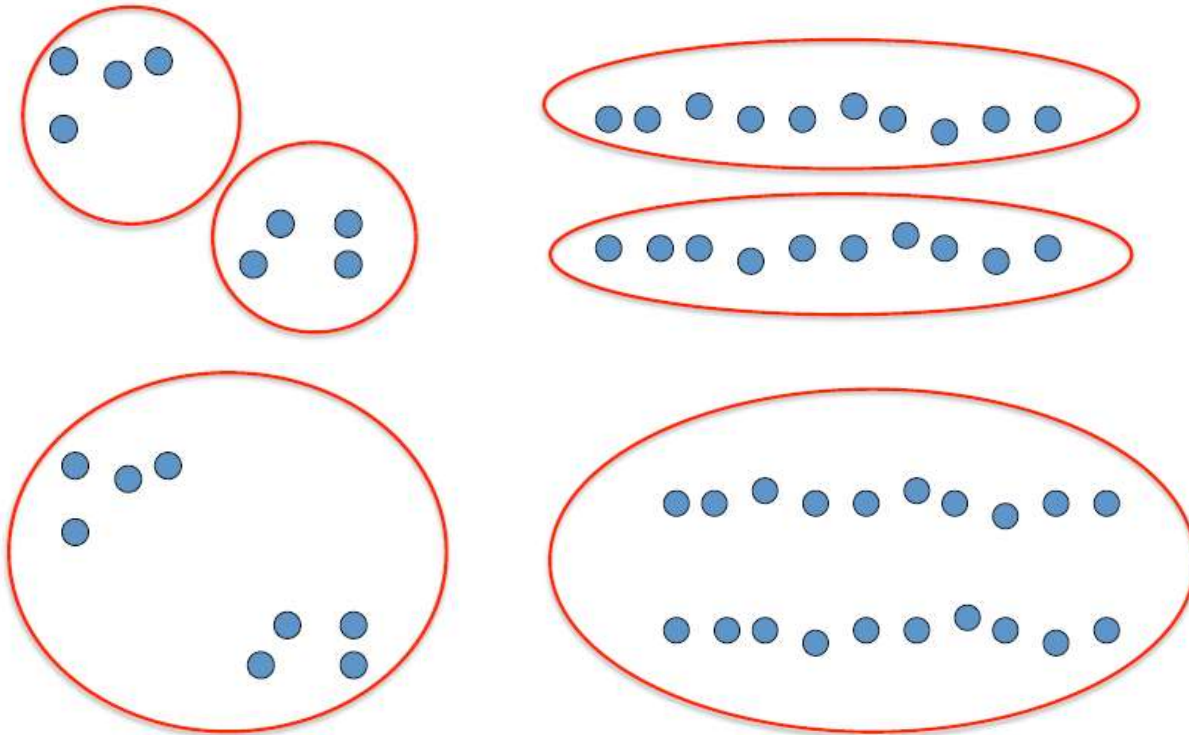


Break up the image into similar regions **without training data**

Clustering: Clustering results are **dependent** on the **measure of similarity** (or **distance**) **between “points”** to be clustered

Basic idea: group together similar instances

Example: 2D point patterns



Clustering

The classification of objects into different groups.

The partitioning of a data set into subsets (clusters), so that the data in each subset (ideally) share some common trait – often according to some defined distance measure.

Euclidean distance

$$d(p_1, p_2) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Clustering: Clustering results are **dependent** on the **measure of similarity** (or **distance**) **between “points”** to be clustered

The objective of K-Means clustering is to minimize total intra-cluster variance, or, the squared error function:

The diagram shows the mathematical formula for the squared error function in K-Means clustering, with several annotations and arrows explaining its components:

- number of clusters**: An arrow points from this text to the variable k in the first summation.
- number of cases**: An arrow points from this text to the variable n in the second summation.
- case i** : An arrow points from this text to the index i in the term $x_i^{(j)}$.
- centroid for cluster j** : An arrow points from this text to the variable c_j .
- Distance function**: A bracket is placed under the expression $\|x_i^{(j)} - c_j\|$ within the norm, with an arrow pointing to it from below.
- objective function**: An arrow points from this text to the variable J on the left side of the equation.

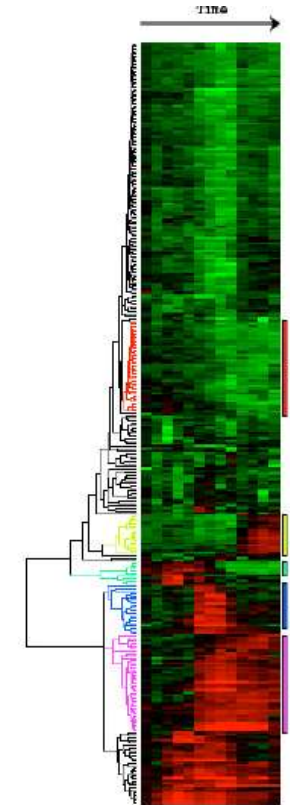
$$J = \sum_{j=1}^k \sum_{i=1}^n \|x_i^{(j)} - c_j\|^2$$

Clustering Examples:

- Document/Image/Webpage Clustering
- Image Segmentation (clustering pixels)



- Clustering web-search results
- Clustering (people) nodes in (social) networks/graphs
- .. and many more..

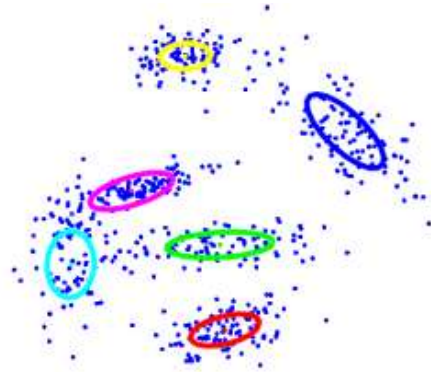


Clustering expression gene data

Eisen et al, PNAS 1998<

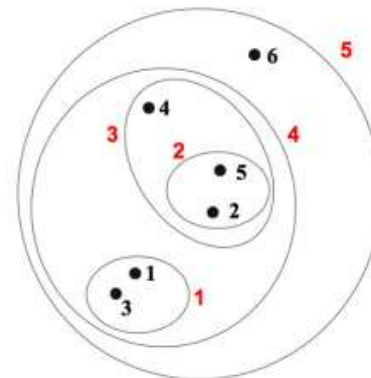
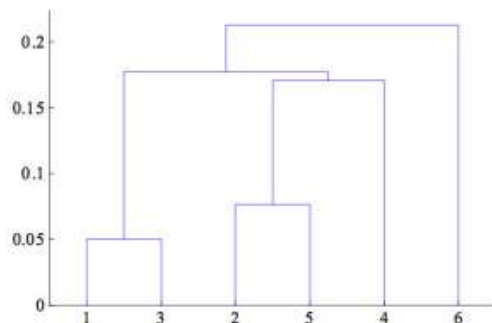
Types of Clustering:

- Flat or Partitional clustering
 - Partitions are independent of each other



In hierarchical clustering, we can look at a clustering for any given number of cluster by “cutting the dendrogram at an appropriate level (so K does not have to be specified)

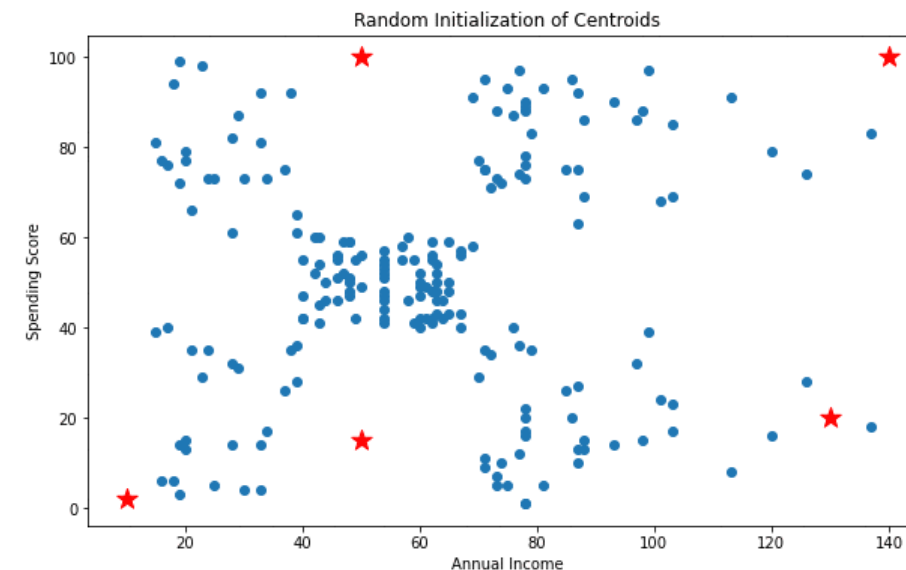
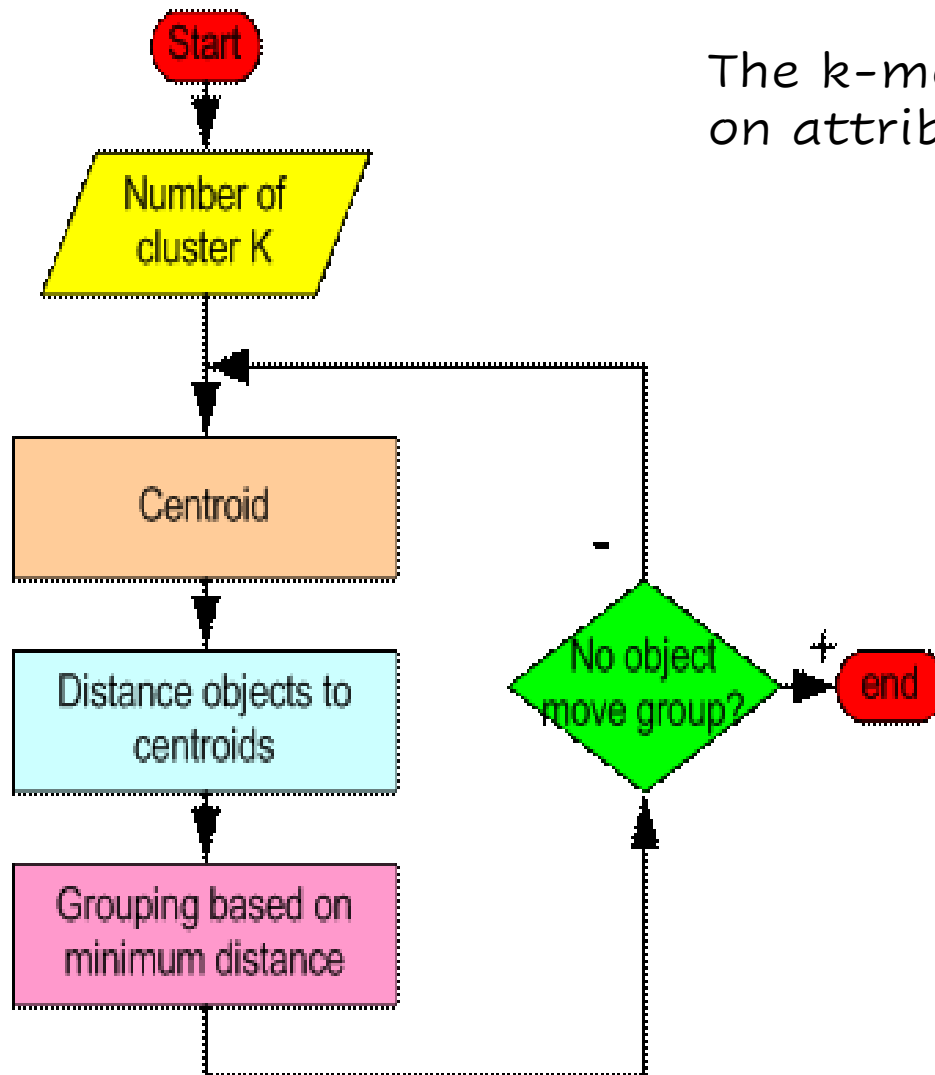
- Hierarchical clustering
 - Partitions can be visualized using a tree structure (a dendrogram)



Hierarchical clustering gives us a clustering at multiple levels of granularity

K-Means FlowChart:

The k-means algorithm is to cluster n objects based on attributes into k partitions, where $k < n$.



K-Means Algorithm: Convergence

- Cluster means don't change.
- Cluster "Loss" doesn't change much

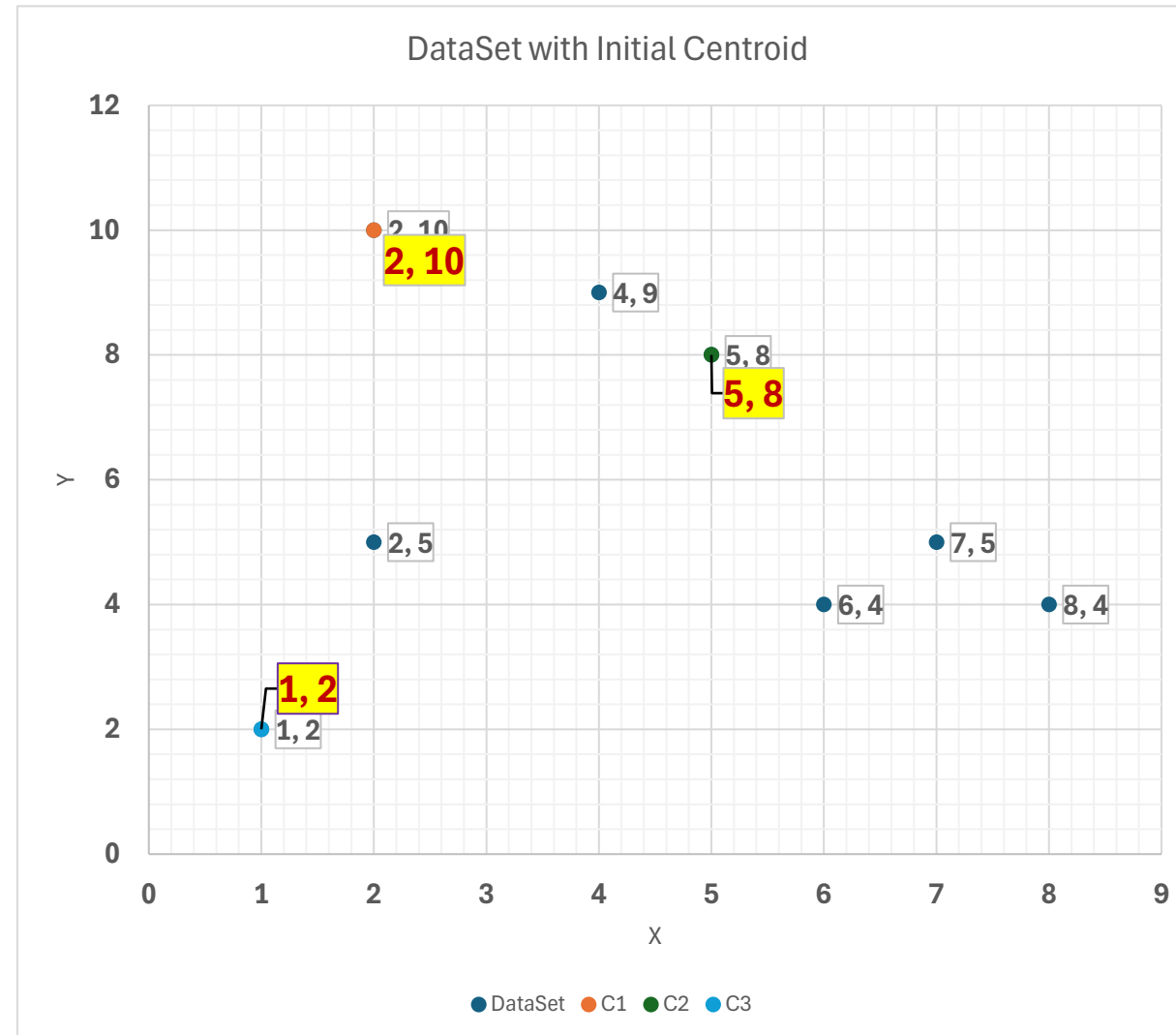
Some ways to declare convergence if between two successive iterations:



K-Mean: Dataset and Plotting

Data Set	
2	10
2	5
8	4
5	8
7	5
6	4
1	2
4	9

C1	2	10
C2	5	8
C3	1	2



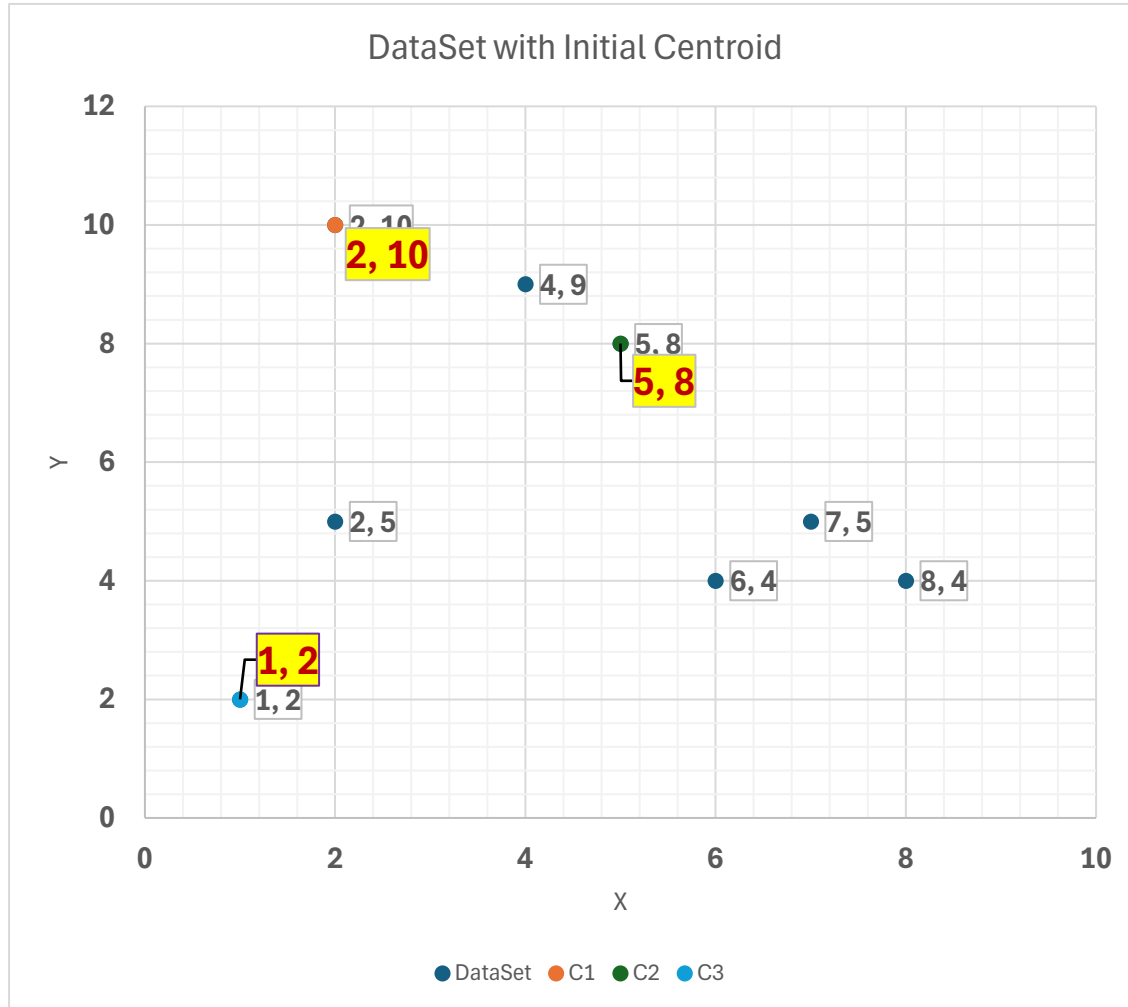


K-Mean: Calculation – Iteration-One

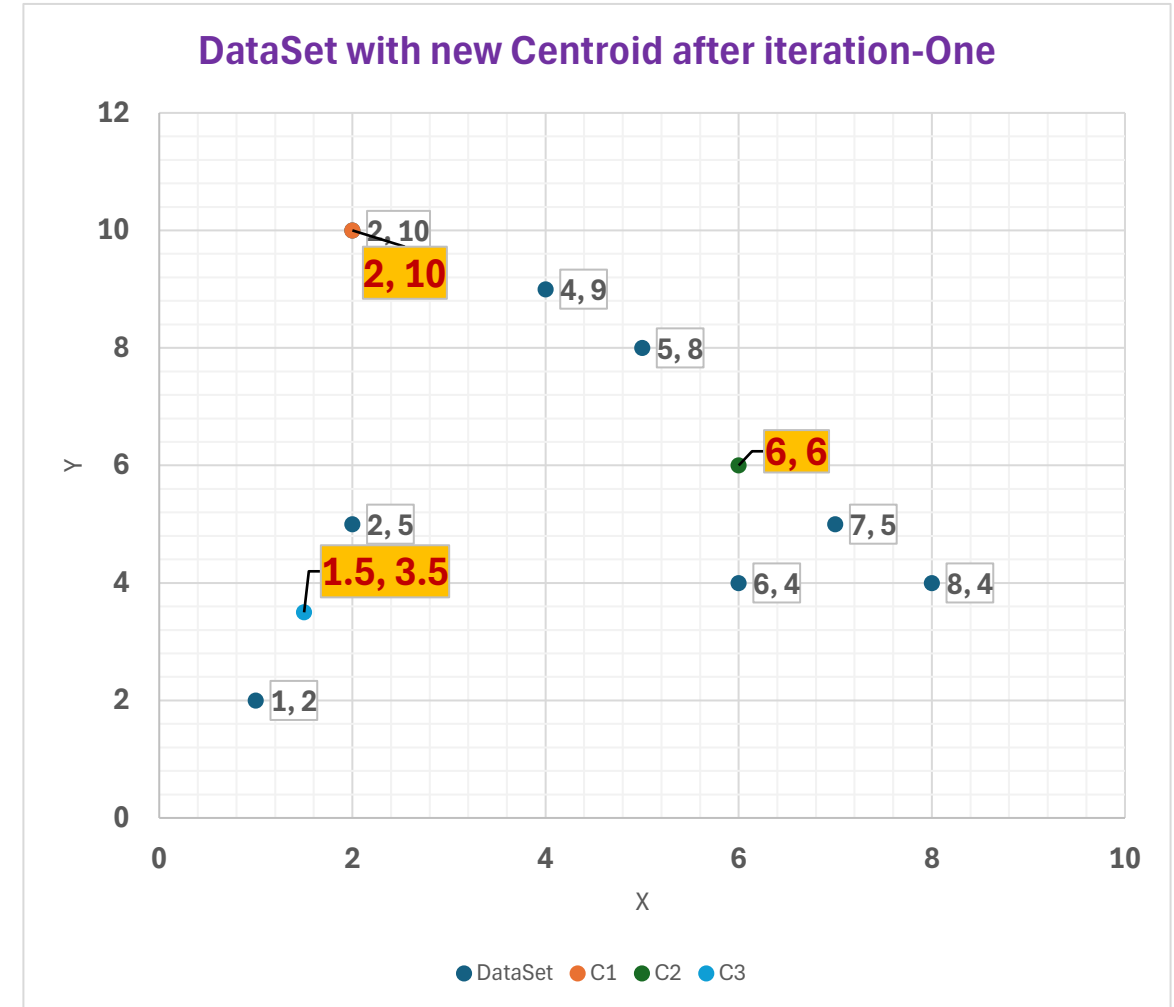
Data Set		C1	C2	C3	Similarity(C1,C2,C3)	Cluster	DataSet			Average(X)	Average(Y)
2	10	0.00	3.61	8.06	0.00	C1	2	10		2	10
2	5	5.00	4.24	3.16	3.16	C3	2	5			
8	4	8.49	5.00	7.28	5.00	C2	8	4			
5	8	3.61	0.00	7.21	0.00	C2	5	8	6	6	
7	5	7.07	3.61	6.71	3.61	C2	7	5			
6	4	7.21	4.12	5.39	4.12	C2	6	4			
1	2	8.06	7.21	0.00	0.00	C3	1	2	1.5	3.5	
4	9	2.24	1.41	7.62	1.41	C2	4	9			
C1	2	10				C1 (Avg)	2	10			
C2	5	8				C2(Avg)	6	6			
C3	1	2				C3(Avg)	1.5	3.5			



K-Mean: Centroid Shifting after Iteration-One



Centroid *before* Iteration-One



Centroid *after* Iteration-One



K-Mean: Centroid Shifting after Iteration-Two

			C1	C2	C3	Similarity(C1,C2,C3)	Old	New			Average(X)	Average(Y)
2	10		0.00	5.66	6.52	0.00	C1	C1	2	10	3.5	9
2	5		5.00	4.12	1.58	1.58	C3	C3	2	5		
8	4		8.49	2.83	6.52	2.83	C2	C2	8	4		
5	8		3.61	2.24	5.70	2.24	C2	C1	5	8	6.5	5.25
7	5		7.07	1.41	5.70	1.41	C2	C2	7	5		
6	4		7.21	2.00	4.53	2.00	C2	C2	6	4		
1	2		8.06	6.40	1.58	1.58	C3	C3	1	2	1.5	3.5
4	9		2.24	3.61	6.04	2.24	C2	C2	4	9		

C1 2 10
C2 6 6
C3 1.5 3.5

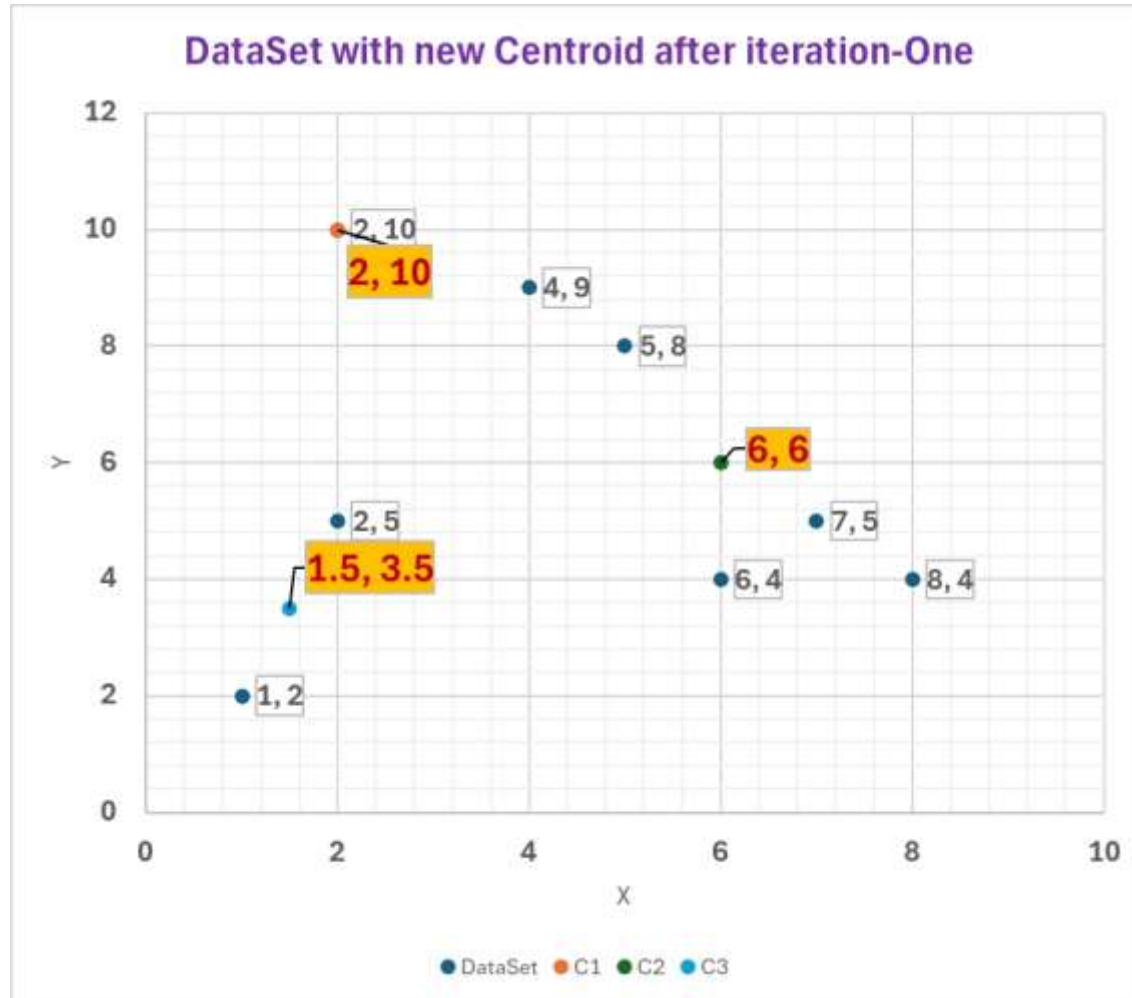
Old
Centr
oid

C1 (Avg) 3.00 9.50
C2(Avg) 6.50 5.25
C3(Avg) 1.50 3.50

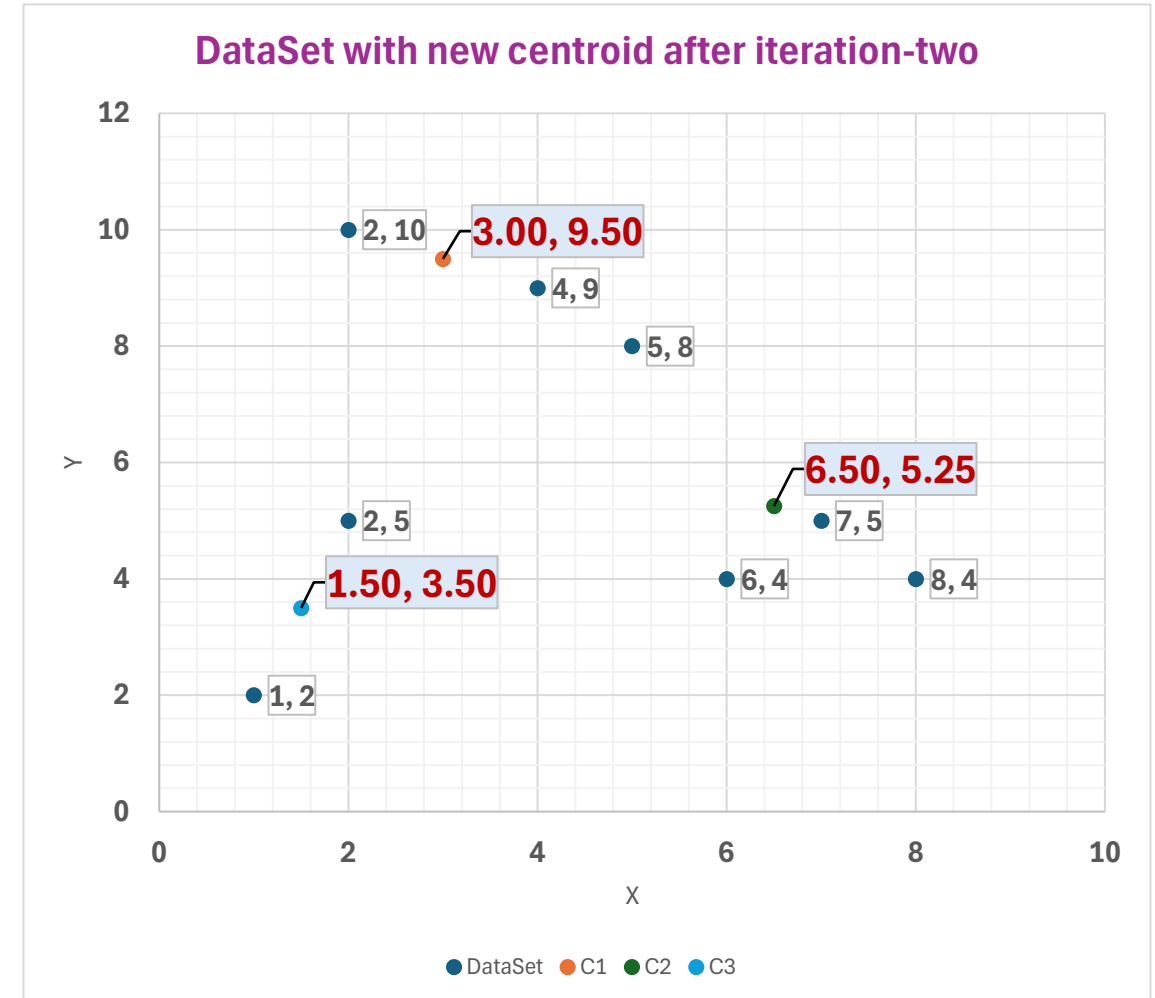
New
Centr
oid



K-Mean: Centroid Shifting after Iteration-Two



Centroid *before* Iteration-Two



Centroid *after* Iteration-Two



K-Mean: Centroid Shifting after Iteration-Three

		C1	C2	C3	Similarity(C1,C2,C3)					Average(X)	Average(Y)
2	10	1.12	6.54	6.52	1.12	C1	C1	2	10	3.67	9.00
2	5	4.61	4.51	1.58	1.58	C3	C3	2	5		
8	4	7.43	1.95	6.52	1.95	C2	C2	8	4		
5	8	2.50	3.13	5.70	2.50	C2	C1	5	8	7.00	4.33
7	5	6.02	0.56	5.70	0.56	C2	C2	7	5		
6	4	6.26	1.35	4.53	1.35	C2	C2	6	4		
1	2	7.76	6.39	1.58	1.58	C3	C3	1	2	1.50	3.50
4	9	1.12	4.51	6.04	1.12	C2	C1	4	9		

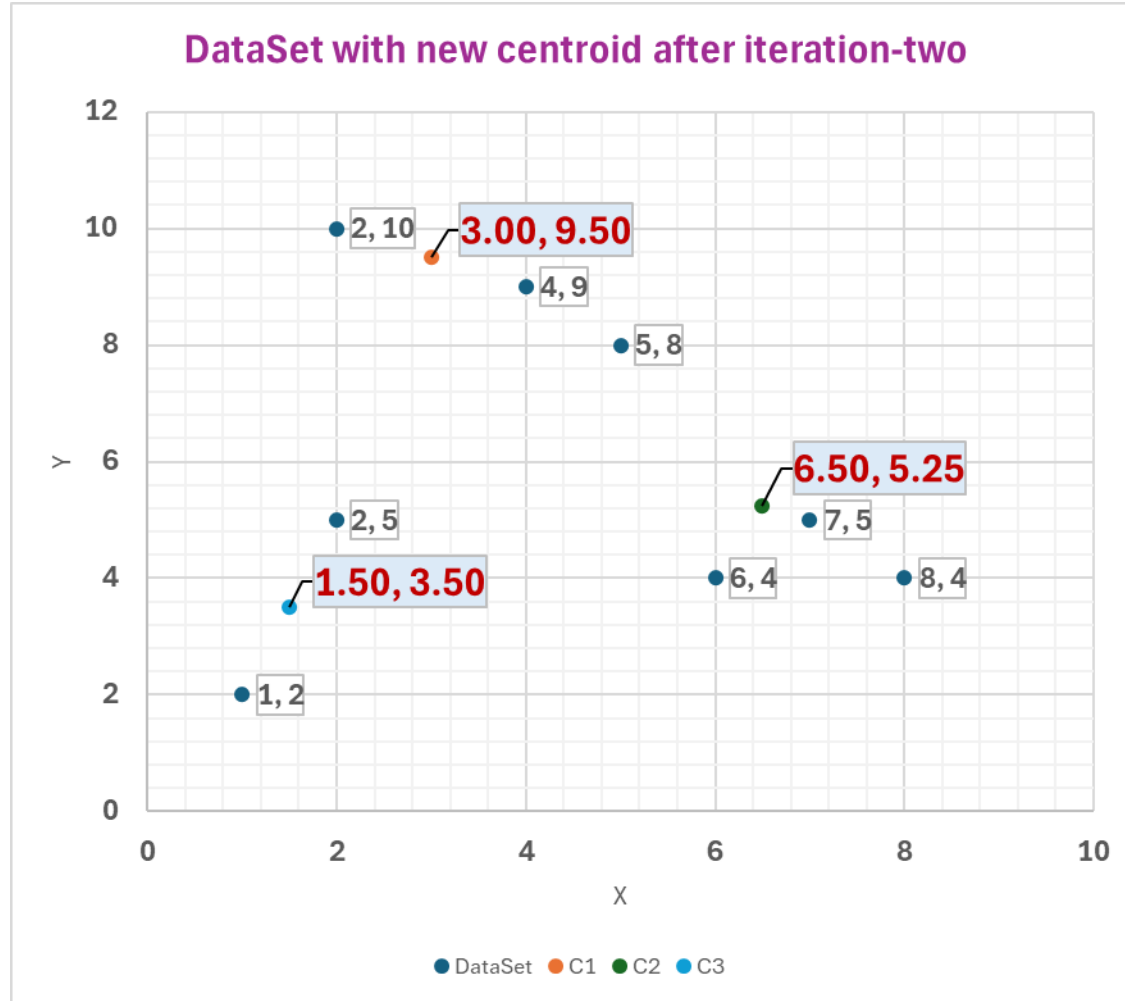
C1	3.00	9.50
C2	6.50	5.25
C3	1.50	3.50

Old Centroid

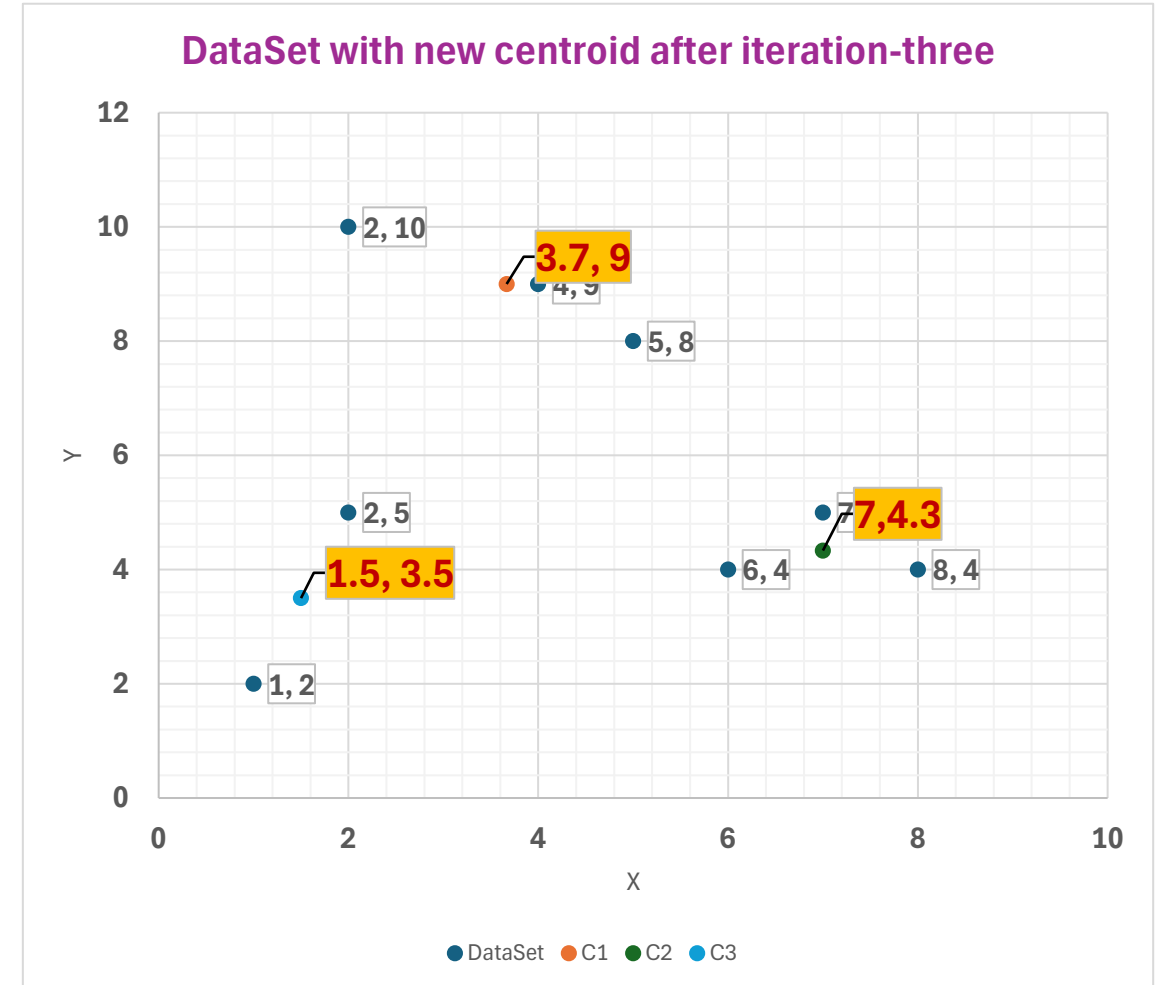
C1 (Avg)	3.67	9
C2(Avg)	7	4.33
C3(Avg)	1.5	3.5

New Centroid

K-Mean: Centroid Shifting after Iteration-Three



Centroid *before* Iteration-Three



Centroid *after* Iteration-Three

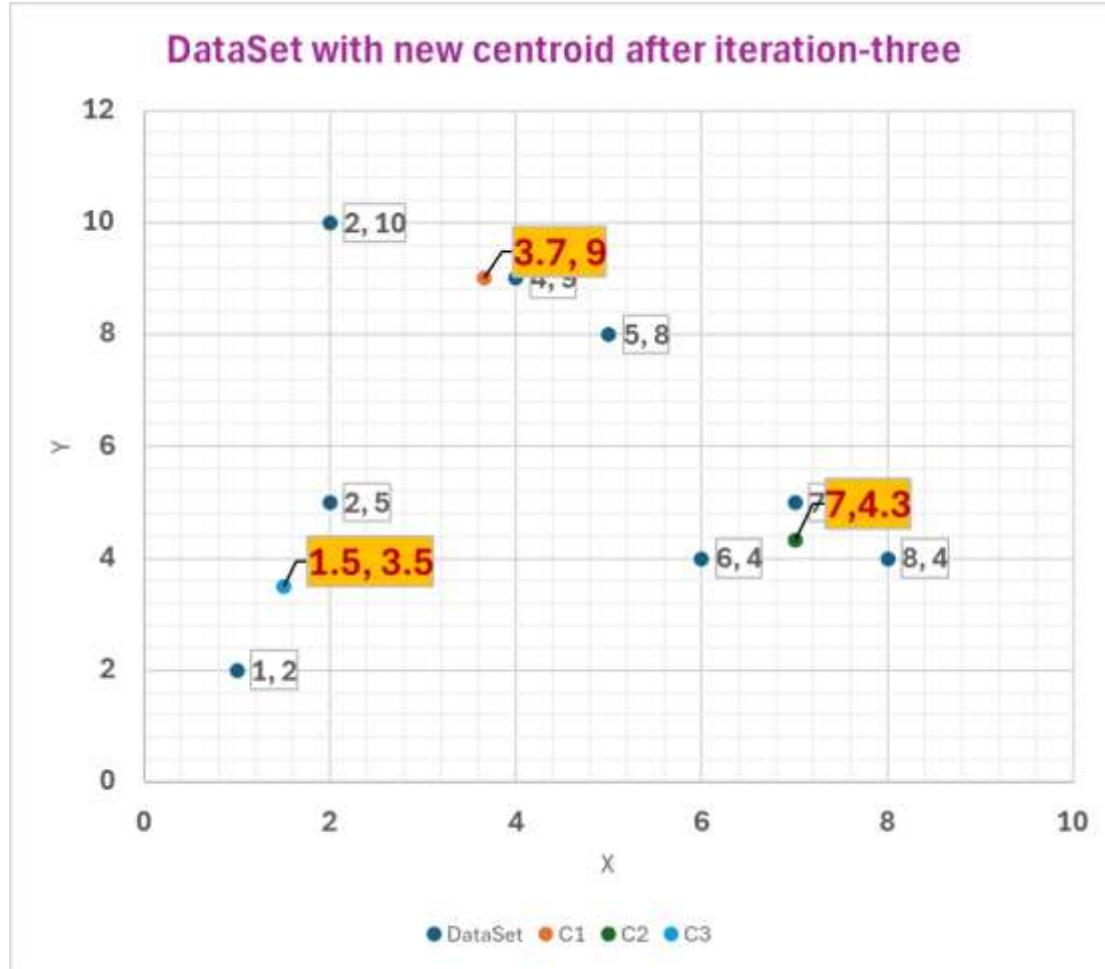


K-Mean: Centroid Shifting after Iteration-Four

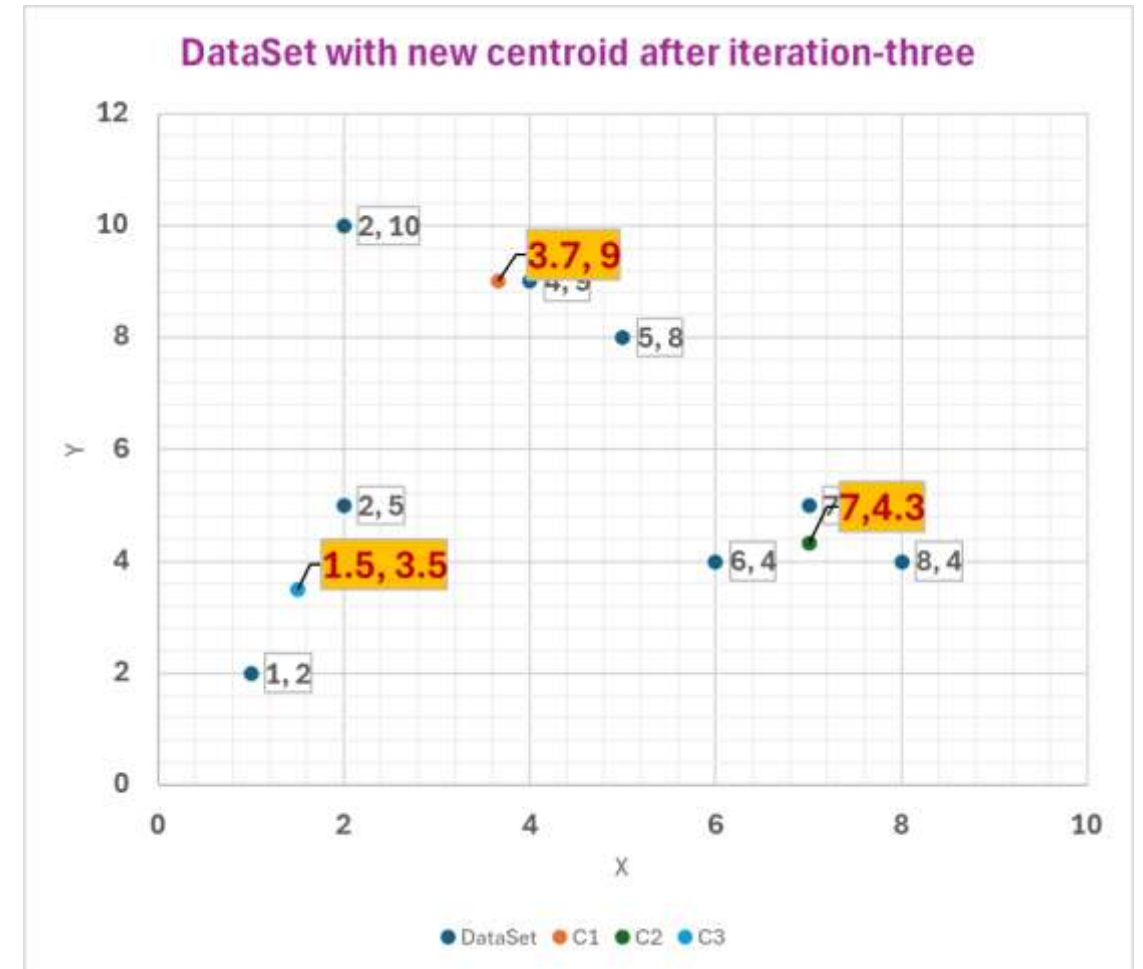
			C1	C2	C3	Similarity(C1,C2,C3)						Average(X)	Average(Y)			
2	10		1.94	7.56	6.52	1.94	C1	C1	2	10		3.67	9.00			
2	5		4.33	5.04	1.58	1.58	C3	C3	2	5						
8	4		6.62	1.05	6.52	1.05	C2	C2	8	4						
5	8		1.67	4.18	5.70	1.67	C1	C1	5	8	7.00	4.33				
7	5		5.21	0.67	5.70	0.67	C2	C2	7	5						
6	4		5.52	1.05	4.53	1.05	C2	C2	6	4						
1	2		7.49	6.44	1.58	1.58	C3	C3	1	2	1.50	3.50				
4	9		0.33	5.55	6.04	0.33	C1	C1	4	9						
C1	3.67	9							C1 (Avg)	3.67	9					
C2	7	4.33							C2(Avg)	7	4.33					
C3	1.5	3.5							C3(Avg)	1.5	3.5					
Old Centr oid														New Centr oid		



K-Mean: Centroid Shifting after Iteration-Four



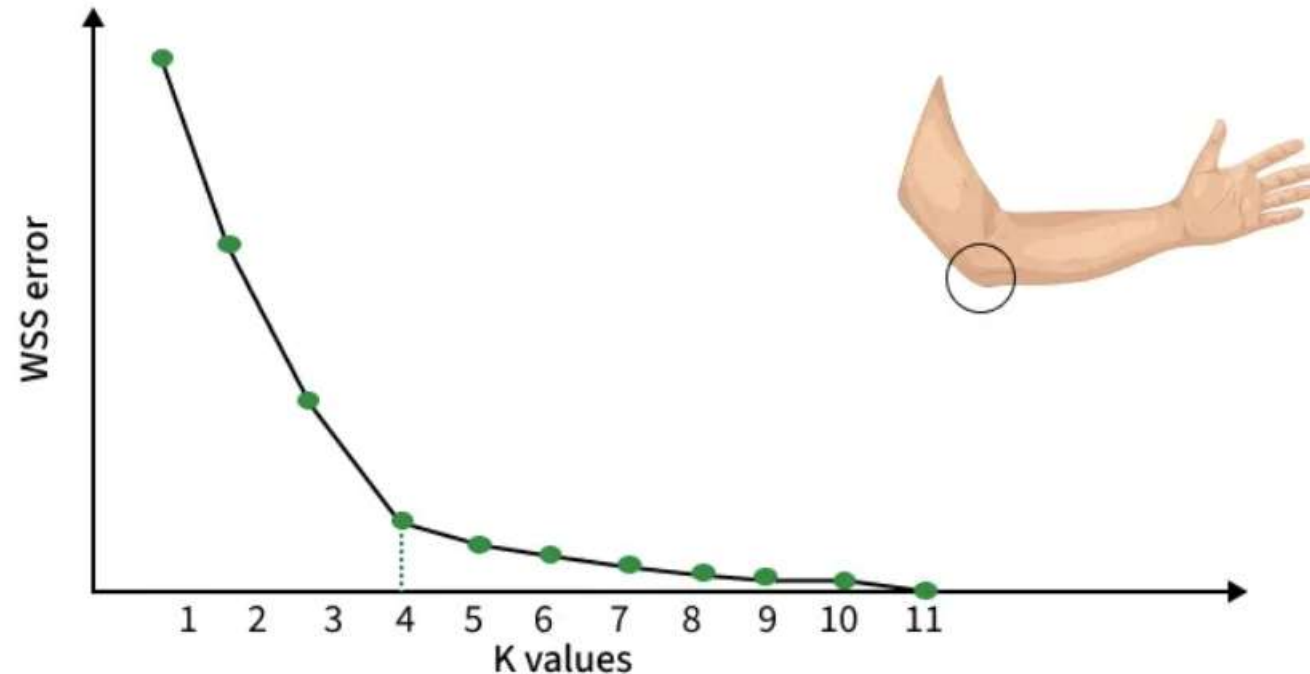
Centroid *before* Iteration-four



Centroid *after* Iteration-four

Elbow Method for optimal value of k in KMeans

The Elbow Method helps by plotting the Within-Cluster Sum of Squares (WCSS) against increasing k values and looking for a point where the improvement slows down, this point is called the "elbow."





Elbow Method for optimal value of k in KMeans

We begin by selecting a range of k values (for example, 1 to 10).

For each k, we run K-Means and calculate WCSS (Within-Cluster Sum of Squares), which shows how close the data points are to their cluster centroids:

$$WCSS = \sum_{i=1}^k \sum_{j=1}^{n_i} \text{distance}(x_j^{(i)}, c_i)^2$$

Where $\text{distance}(x_j^{(i)}, c_i)$ represents the distance between the j^{th} data point $x_j^{(i)}$ in cluster i and the centroid c_i of that cluster.

After computing WCSS for all k values, we plot k vs WCSS.

WCSS always decreases as k increases because more clusters reduce the internal spread.

However, after a certain point, the improvement becomes very small. This bend or “elbow” in the curve indicates the point where adding more clusters no longer gives meaningful improvement.

the principle of similarity where it predicts the label or value of a new data point by considering the labels or values of its K nearest neighbors

K-Nearest Neighbor (KNN) Algorithm - Classification

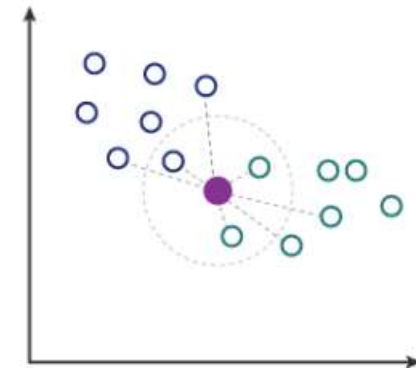
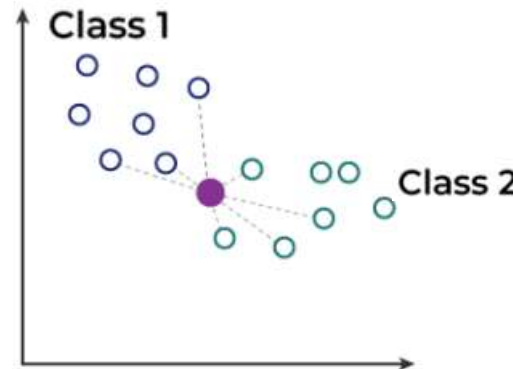
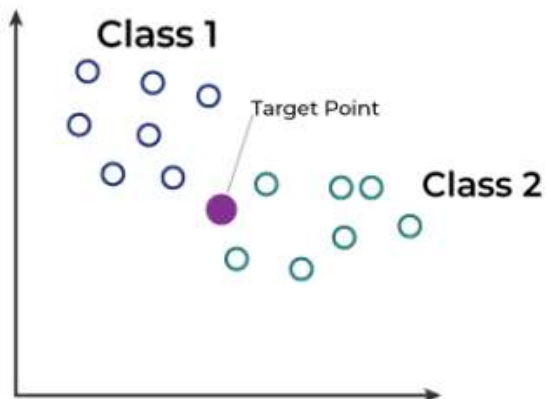
It is a supervised machine learning algorithm generally used for classification but can also be used for regression tasks.

Step 1:

It works by finding the "k" closest data points (neighbors) to a given input

Step 2:

It makes a predictions based on the majority class (for classification) or the average value (for regression).



Properties of KNN:

Simple to use:

Easy to understand and implement.

No training step:

No need to train as it just stores the data and uses it during prediction.

Few parameters:

Only needs to set the number of neighbors (k) and a distance method.

Versatile:

Works for both classification and regression problems.

Slow with large data:

Needs to compare every point during prediction.

Struggles with many features:

Accuracy drops when data has too many features.

Can Overfit:

It can overfit especially when the data is high-dimensional or not clean.

Applications of KNN:

Recommendation Systems:

Suggests items like movies or products by finding users with similar preferences.

Spam Detection:

Identifies spam emails by comparing new emails to known spam and non-spam examples.

Customer Segmentation:

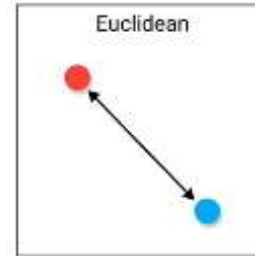
Groups customers by comparing their shopping behavior to others.

Speech Recognition:

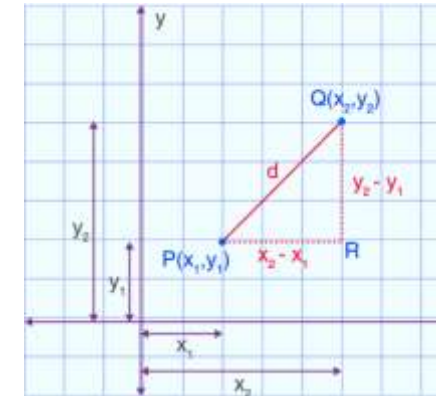
Matches spoken words to known patterns to convert them into text.

K-Nearest Neighbor (KNN) Predict the Class labels

Sepal length (cm)	Sepal width (cm)	Species
5.3	3.7	Setosa
5.1	3.8	Setosa
7.2	3	Virginica
5.4	3.4	Setosa
5.1	3.3	Setosa
5.4	3.9	Setosa
7.4	2.8	Veriscicolor
7.3	2.9	Virginica
6	2.7	Veriscicolor
5.8	2.8	Virginica
6.3	2.3	Veriscicolor
5.1	2.5	Veriscicolor
6.3	2.5	Veriscicolor
5.5	2.4	Veriscicolor



$$d = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$



Using Pythagoras theorem,

$$\text{Hypotenuse}^2 = \text{Base}^2 + \text{Perpendicular}^2$$

$$PQ^2 = PR^2 + QR^2$$

$$\text{Therefore, } d^2 = (x_2 - x_1)^2 + (y_2 - y_1)^2$$

$$d = \sqrt{[(x_2 - x_1)^2 + (y_2 - y_1)^2]}$$

Test data

Sepal length (cm)	Sepal width (cm)	Species
5.2	3.1	??

Computing the Euclidean distance

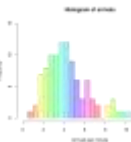
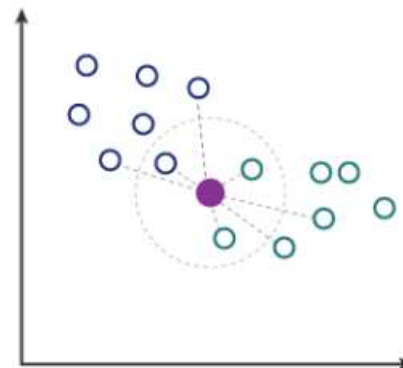
Sepal length (cm)	Sepal width (cm)	Species	Euclidean distance (cm)
5.3	3.7	Setosa	0.61

Distance calculation and Rank Assignment:

Sepal length (cm)	Sepal width (cm)	Species	Euclidean distance (cm)
5.3	3.7	Setosa	0.61
5.1	3.8	Setosa	0.71
7.2	3	Virginica	2.00
5.4	3.4	Setosa	0.36
5.1	3.3	Setosa	0.22
5.4	3.9	Setosa	0.82
7.4	2.8	Versicolor	2.22
7.3	2.9	Virginica	2.11
6	2.7	Versicolor	0.89
5.8	2.8	Virginica	0.67
6.3	2.3	Versicolor	1.36
5.1	2.5	Versicolor	0.61
6.3	2.5	Versicolor	1.25
5.5	2.4	Versicolor	0.76

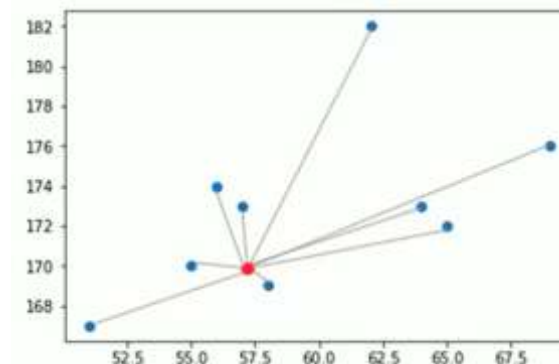
Sort Euclidean distance column in ascending order

Sepal length (cm)	Sepal width (cm)	Species	Euclidean distance (cm)	Rank
5.1	3.3	Setosa	0.22	1
5.4	3.4	Setosa	0.36	2
5.3	3.7	Setosa	0.61	3
5.1	2.5	Versicolor	0.61	4
5.8	2.8	Virginica	0.67	5
5.1	3.8	Setosa	0.71	6
5.5	2.4	Versicolor	0.76	7
5.4	3.9	Setosa	0.82	8
6	2.7	Versicolor	0.89	9
6.3	2.5	Versicolor	1.25	10
6.3	2.3	Versicolor	1.36	11
7.2	3	Virginica	2.00	12
7.3	2.9	Virginica	2.11	13
				14

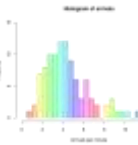
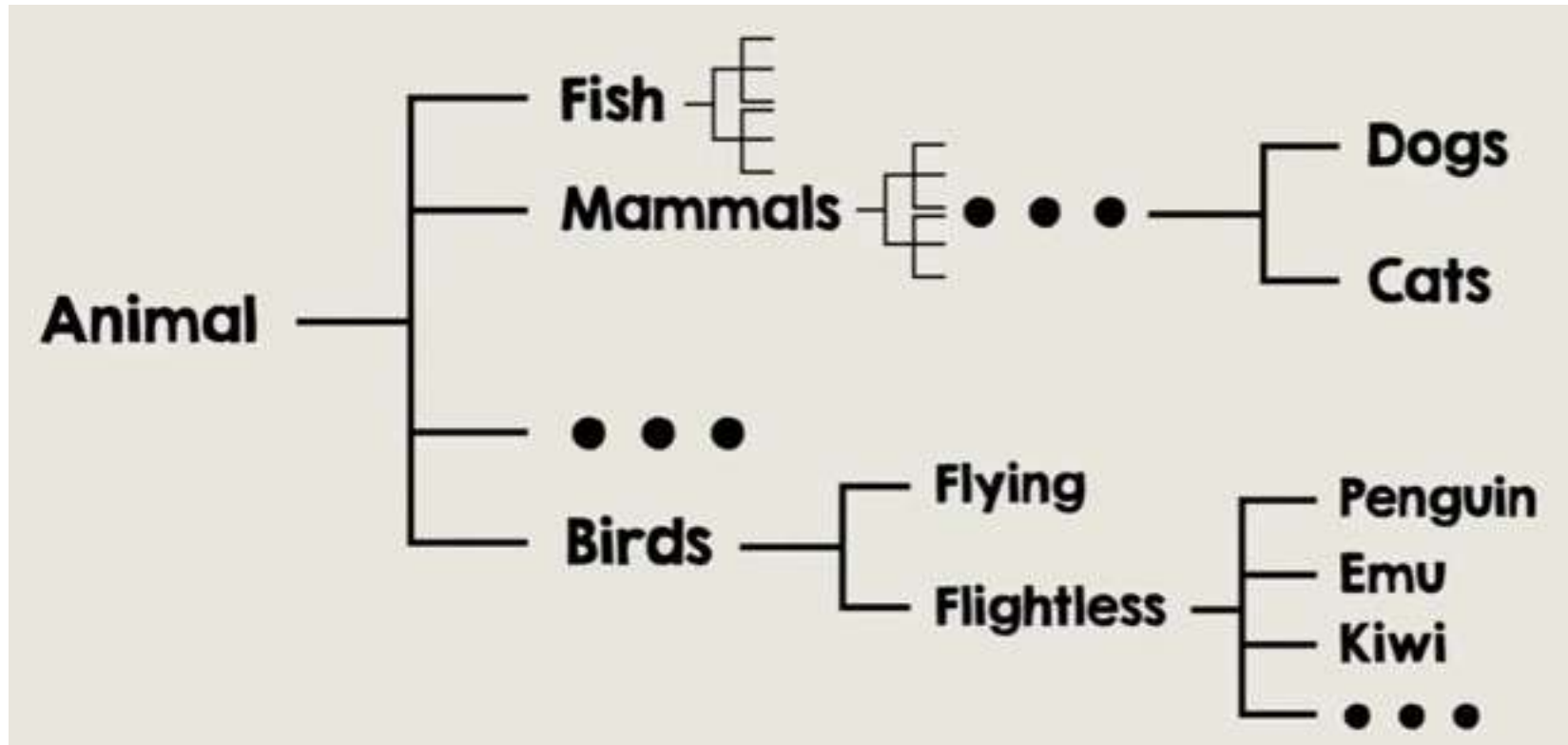


Home work: KNN

Height (cm)	Weight (Kg)	Class
167	51	Underweight
182	62	Normal
176	69	Normal
173	64	Normal
172	65	Normal
174	56	Underweight
169	58	Normal
173	57	Normal
170	55	Normal
170	57	???



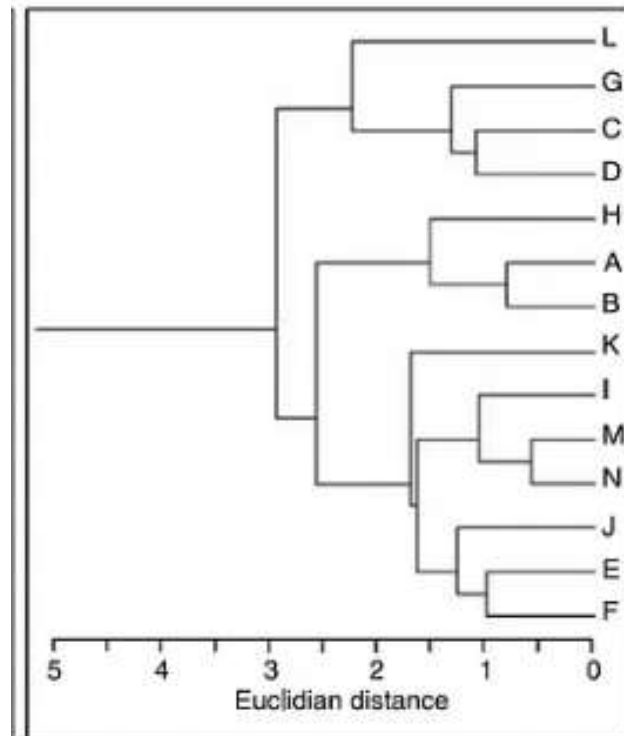
Hierarchical Clustering :Taxonomy of the Animal Kingdom



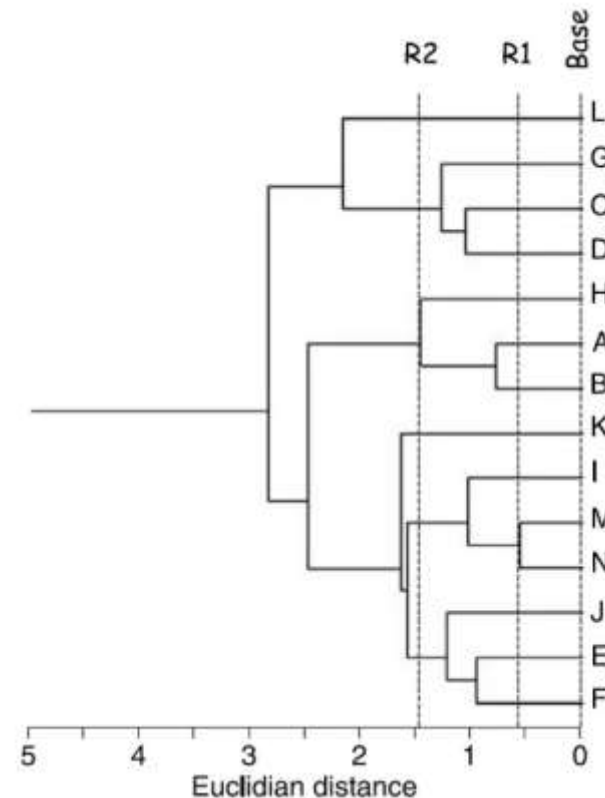
Hierarchical Clustering

A *dendrogram* is a **diagram used to depict the hierarchical relationship between things.**

we assign each object (data point) to a separate cluster. Then compute the distance (similarity) between each of the clusters



Using Reference Lines in a Dendrogram:



the vertical line that connects the clusters in a group to be a reference line that extends down to the horizontal scale.

One can determine how similar its children clusters are by seeing where the reference line intersects the horizontal scale.

The group connected at R1, M to N, are more similar to each other.

R2 line H to clusters A and B.

Why Segmentation Is Essential in Robotics?

Enables robots to interpret complex environments

Supports safe navigation and collision avoidance

Allows precise manipulation of objects

Provides rich scene understanding for decision-making

Forms the basis for many higher-level AI tasks in robotics



Ameca Robot is real

Ameca is the world's most advanced human-shaped robot developed by a UK based company called Engineered Arts.

Ameca is designed to support the testing and development of artificial intelligence and machine learning systems

Why Dendrograms Matter in Computer Vision?

No need to pre-specify number of clusters
(unlike k-means)

Reveal hierarchical structure in visual data

Allow multi-scale interpretation
(fine → coarse)

Help diagnose feature quality and
clustering behavior

Handles complex cluster shapes and nested
structures

Image Segmentation & Image Analysis

hierarchical clustering helps segment images by
grouping pixels or regions based on similarity.

It supports multi-scale segmentation because the
dendrogram can be cut at different levels.

Gene Expression & Bioinformatics

genes with similar expression patterns, helping
identify biological pathways or disease subtypes

Pattern Recognition & Data Exploration

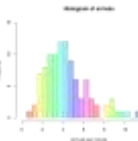
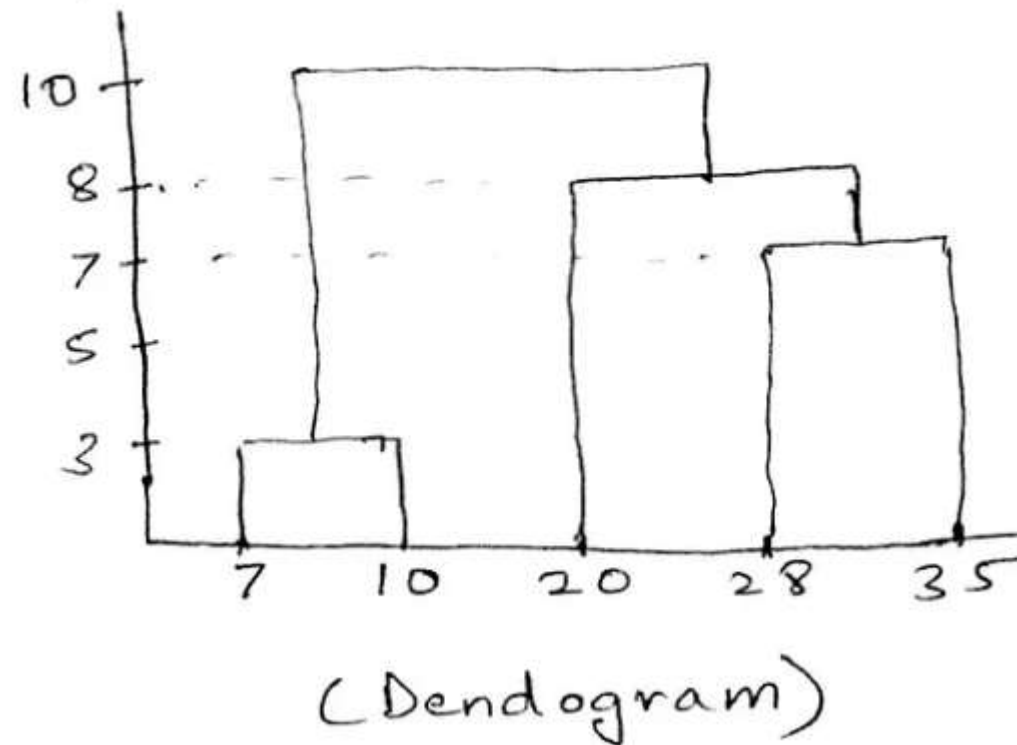
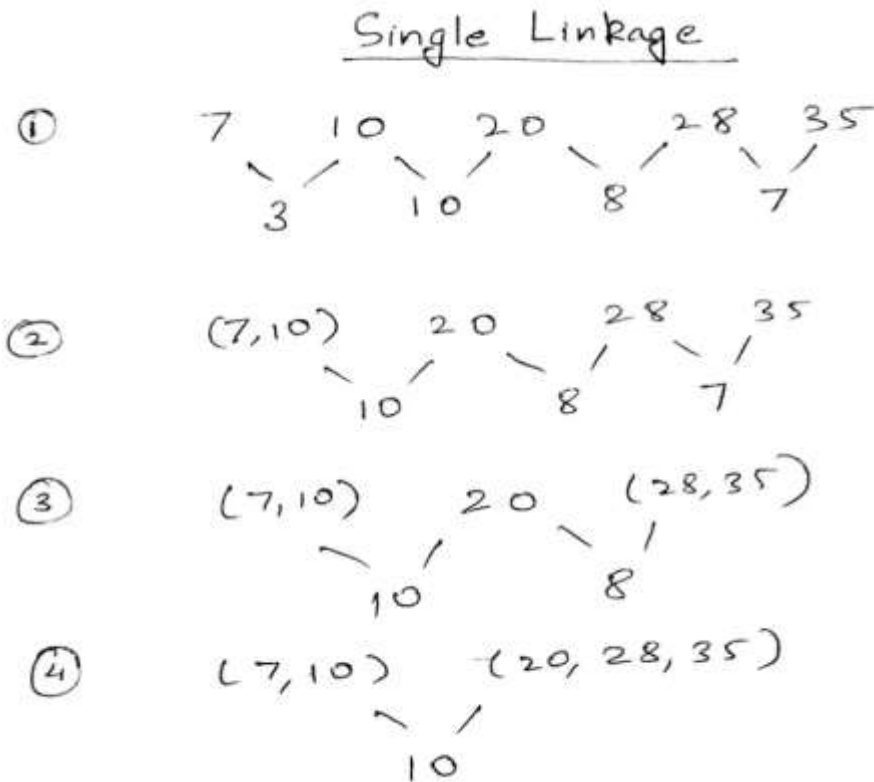
the structure of the data is unknown.

It helps visualize relationships using dendrograms
and uncover nested patterns

Single Linkage:

For the one dimensional data set $\{7,10,20,28,35\}$, perform hierarchical clustering and plot the dendrogram to visualize it.

In single link hierarchical clustering, we **merge** in each step **the two clusters**, whose **two closest members have the smallest distance**.

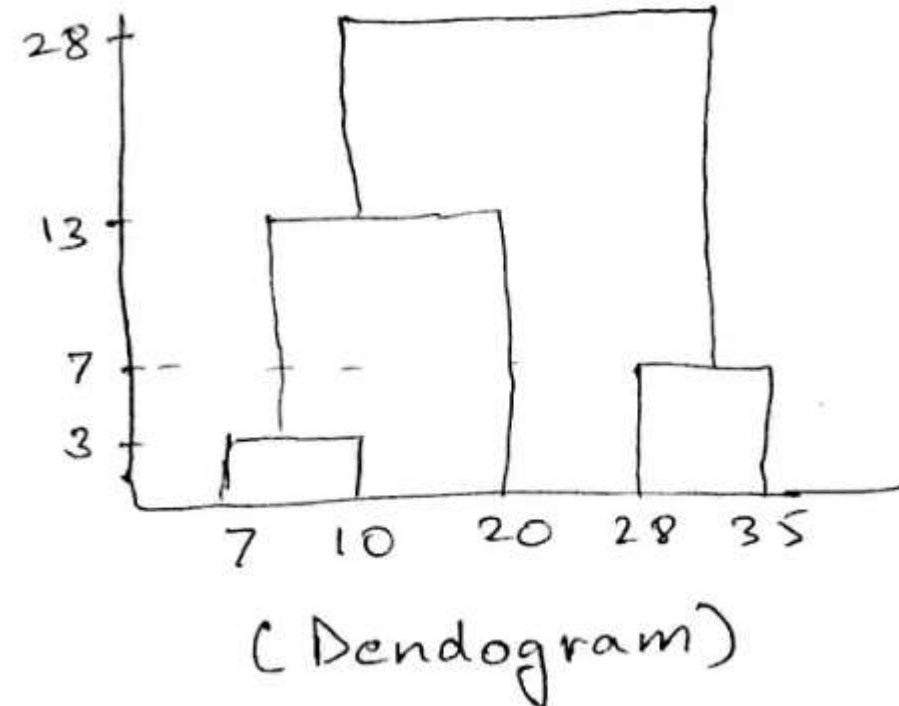
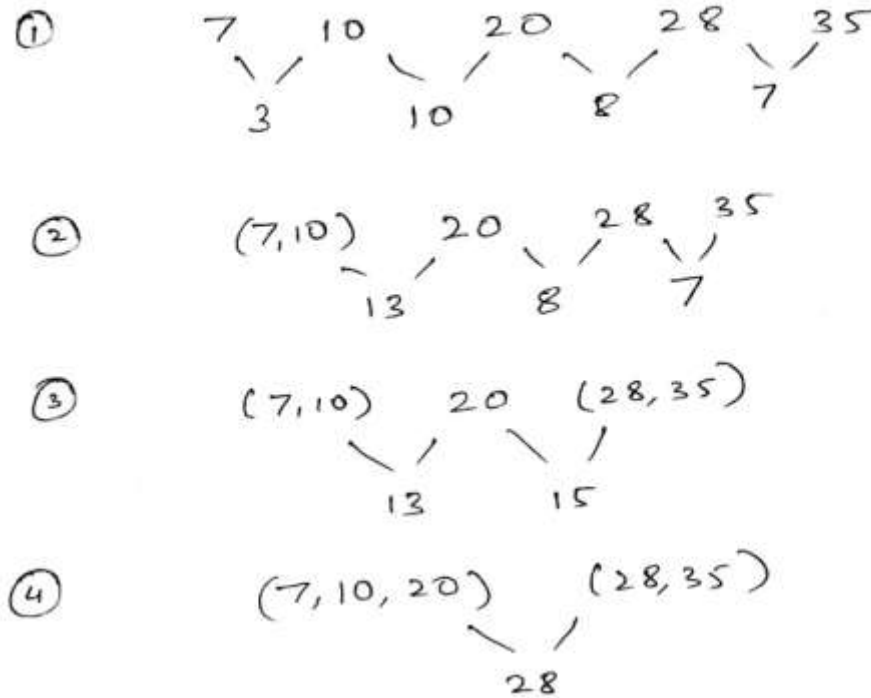


Complete Linkage:

For the one dimensional data set $\{7,10,20,28,35\}$, perform hierarchical clustering and plot the dendrogram to visualize it.

In complete link hierarchical clustering, we merge in the members of the clusters in each step and also the smallest maximum pairwise distance.

Complete Linkage



Otsu's Method Algorithm

It is based on statistical –decision theory

minimizes the in-class variances or maximizes the between class-variance

Class is a set of pixels belonging to a region.

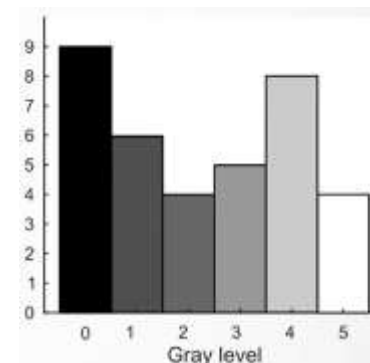
maximizes the between class variance to search global threshold intensity

$$\sigma_B^2 = W_b W_f (\mu_b - \mu_f)^2$$

$W_b W_f$ = Number of pixels in background/foreground / Total number of pixel

$\mu_b \mu_f$ = Mean intensity of background / foreground

0	1	2	1	0	0
0	3	4	4	1	0
2	4	5	5	4	0
1	4	5	5	4	1
0	3	4	4	3	1
0	2	3	3	2	0

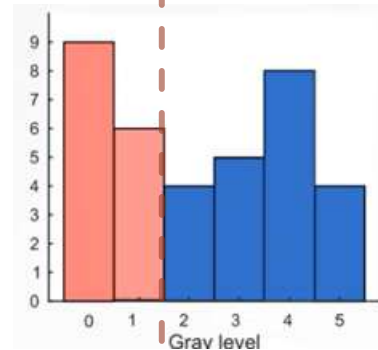


Global Thresholding using Otsu's Algorithm

Background Foreground

$$W_b = \frac{9 + 6}{36} = 0.42$$

$$\mu_b = \frac{(9 \times 0) + (6 \times 1)}{9 + 6} = 0.4$$

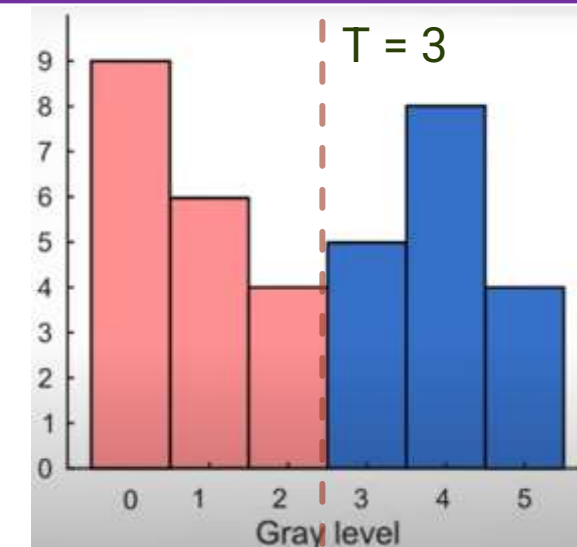
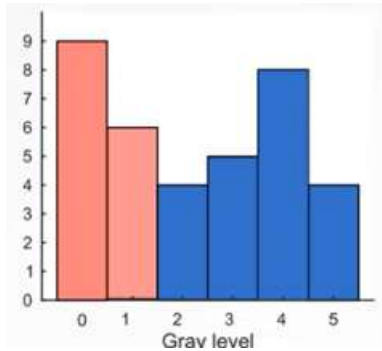


$$W_f = \frac{4 + 5 + 8 + 4}{36} = 0.58$$

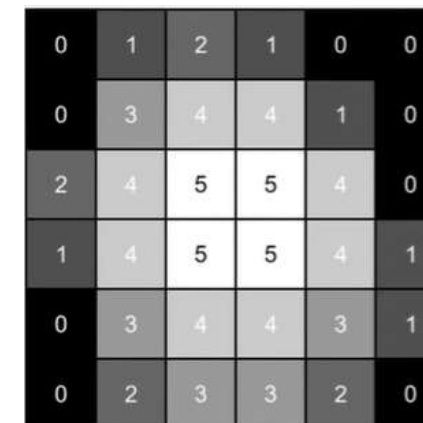
$$\mu_f = \frac{(4 \times 2) + (5 \times 3) + (8 \times 4) + (4 \times 5)}{4 + 5 + 8 + 4} = 3.57$$

$$\sigma_B^2 = W_b W_f (\mu_b - \mu_f)^2 = 2.44$$

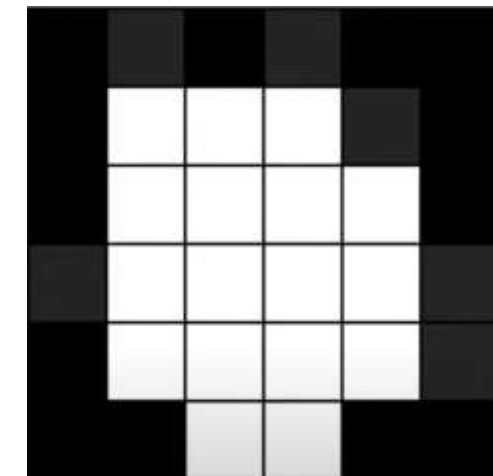
Otsu's Algorithm: InterClass Variance for different GrayLevels



I_t	0	1	2	3	4	5
W_b	0	0.25	0.42	0.53	0.67	0.89
μ_b	0	0	0.40	0.74	1.21	1.91
W_f	1	0.75	0.58	0.47	0.33	0.11
μ_f	2.25	3.00	3.57	3.94	4.33	5.00
σ_b^2	0	1.69	2.44	2.56	2.17	0.95



Original Image



Output Image

Otsu's Results: Optimum Global Thresholding



Original image



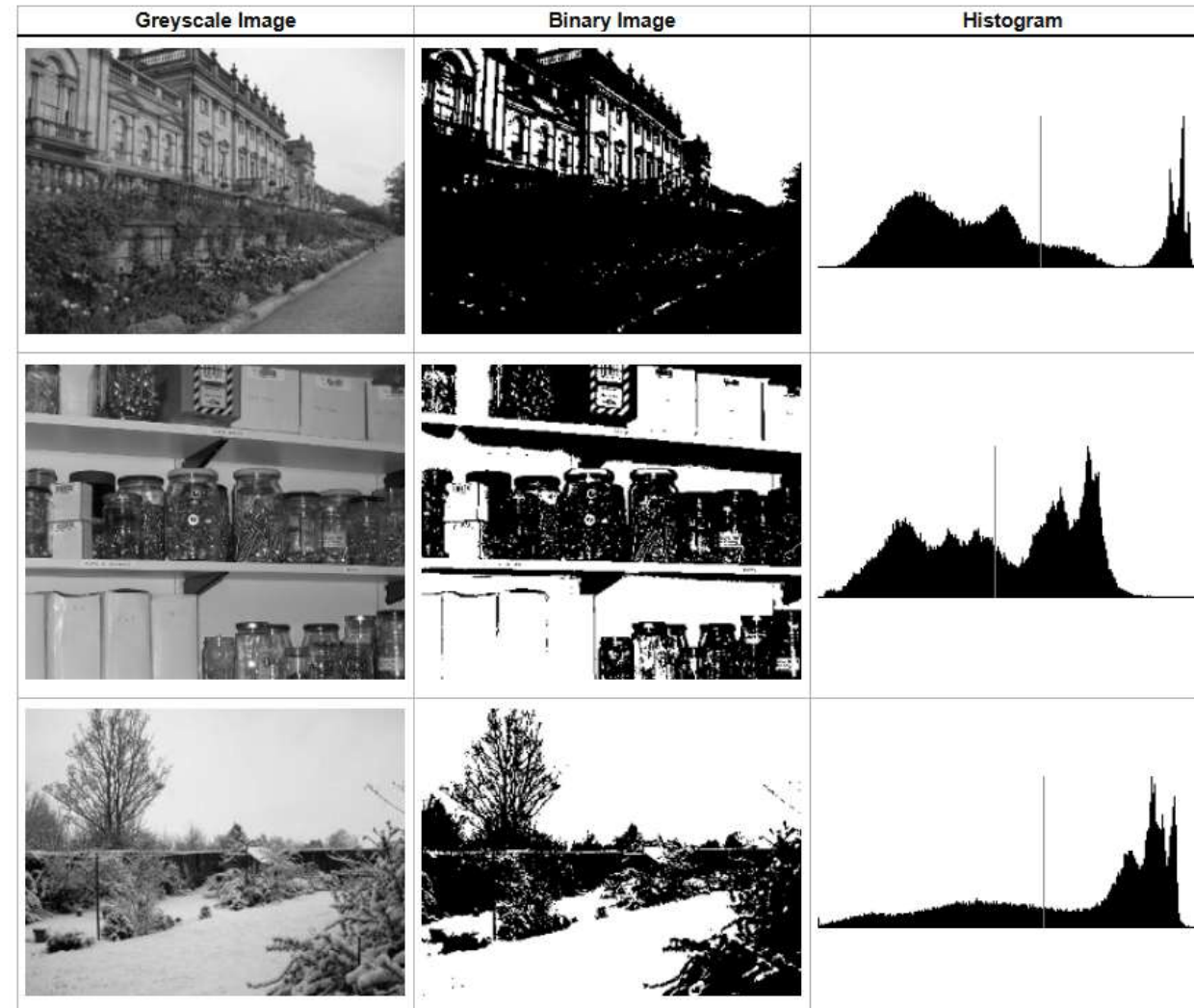
image thresholding using Otsu's algorithm

Otsu's **Thresholding** performs well on bimodal distribution with a deep and sharp valley between two peaks.

This method is not suited for images:

Small object size | noisy images | in homogenous lighting conditions

Otsu Method: Examples



Mean based v/s Otsu based approaches

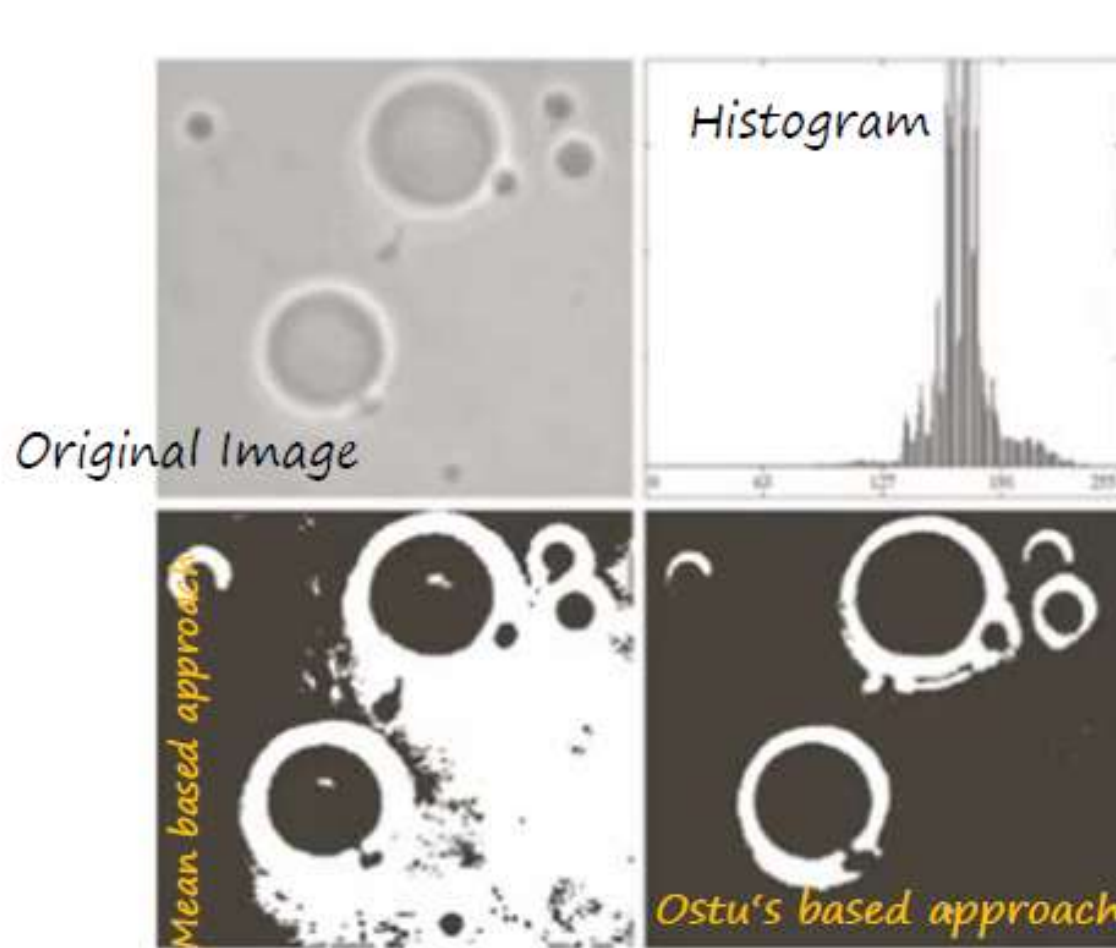
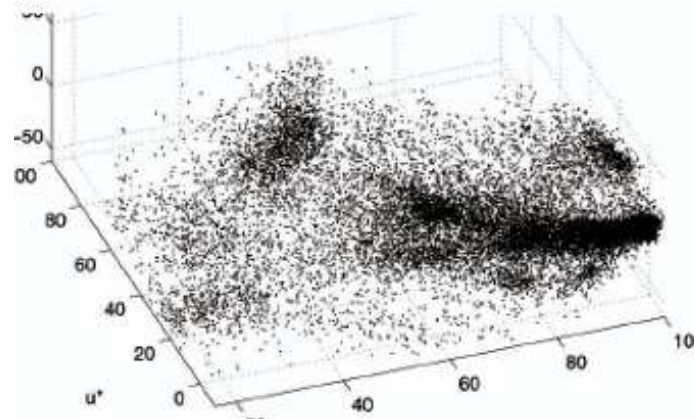


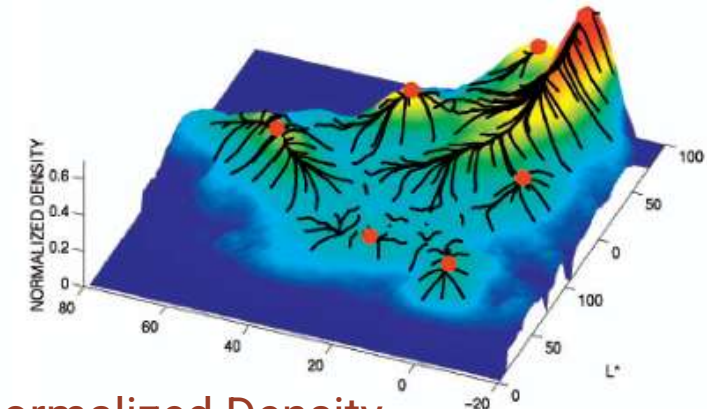
FIGURE 10.39
(a) Original image.
(b) Histogram (high peaks were clipped to highlight details in the lower values).
(c) Segmentation result using the basic global algorithm from Section 10.3.2.
(d) Result obtained using Otsu's method. (Original image courtesy of Professor Daniel A. Hammer, the University of Pennsylvania.)

Mean Shift Segmentation or Mean Shift Clustering

Try to find modes of a non-parametric density



Pixel feature distribution



Normalized Density

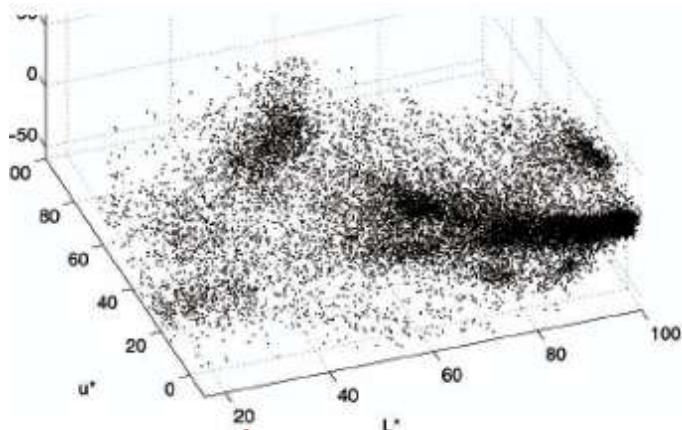
Each hill represent a cluster

Peak (mode) of hill represents cluster "center"

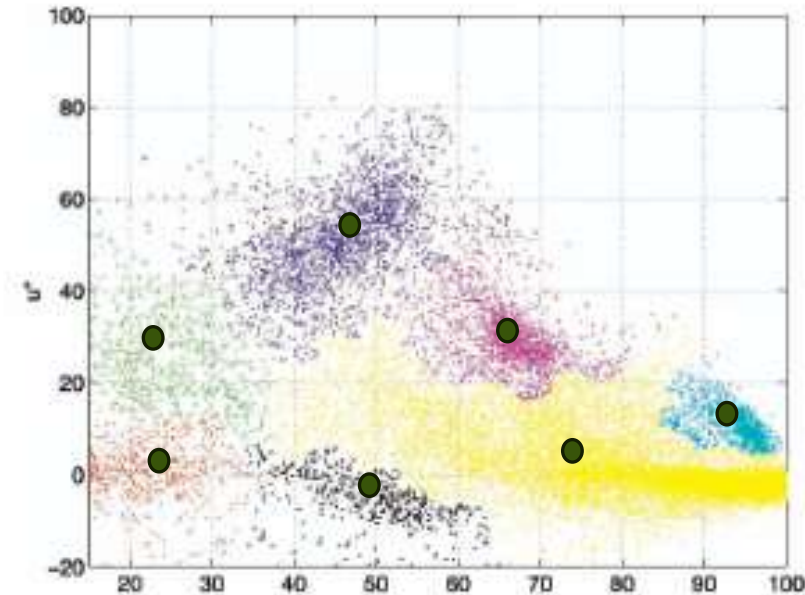
Each pixel climbs the steepest hill within its neighborhood

Pixel assigned to the hill (cluster) it climbs.

Image Segmentation using Mean Shift Algorithm



Pixel feature distribution



§ Non-parametric approach to density estimation (“how many data points are in a certain region?”)

§ Find the local modes of this density

§ All points that “belong” or “lead” to the same mode form a cluster

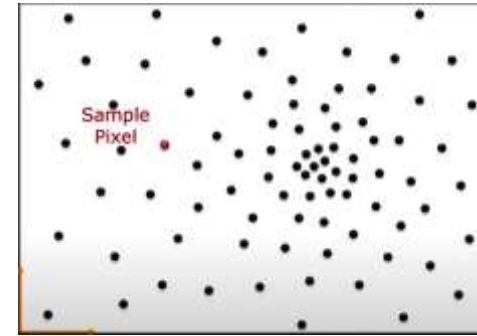
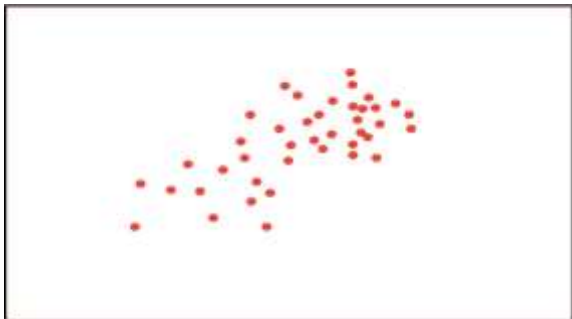
Hill Climbing using Mean Shift

Mean shift clustering is used to identify clusters in datasets where the number of clusters is not known beforehand

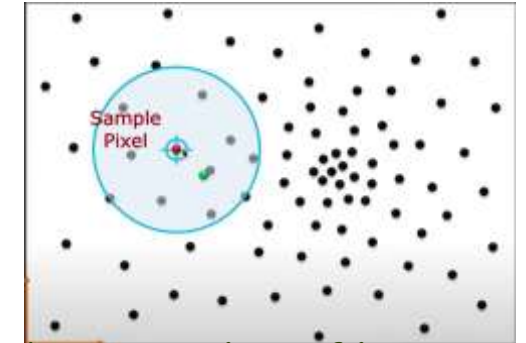
It can segment images into regions based on pixel intensity or color

Algorithm

It finds clusters by iteratively shifting data points toward the densest regions in the feature space



Select a sample pixel.



Select a region of interest (ROI)

Density Estimation:

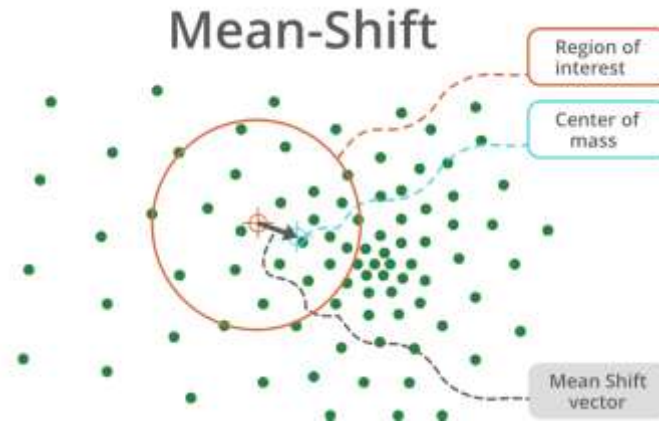
For each data point, define a window around it, called the radius, and then compute the mean of those data points within this radius.

Shifting: Shift each point to this mean position.

This step moves the point towards the region of higher density.

Convergence: Repeat Density estimation and shifting iteratively until convergence.

It implies that the points have stabilized around the local maxima of the density function.



Mean Shift Algorithm Results



Pros

- Good general-practice segmentation
- Flexible in number and shape of regions
- Robust to outliers

Cons

- Have to choose kernel size in advance
- Not well suited for high-dimensional features

<http://www.caip.rutgers.edu/~comanici/MSPAMI/msPamiResults.html>